

From Source to Citation:

Developing and Evaluating a Reliable and Traceable
RAG System with IBM Granite

JIAXI YOU, WEIKAI MAO, MING WEI,
ZENAN WU, ZHANYU XU, MENGXIAO WU, FRIDA CHEN



A dissertation submitted to the University of Bristol in accordance
with the requirements for award of the degree of
MSc in Computer Science in the Faculty of Science and Engineering.

School of Computer Science

September 2026

Abstract

Enterprise information retrieval often requires identifying a small amount of answer-supporting evidence within large collections of similar or irrelevant material. In this needle-in-a-haystack setting, retrieval-augmented generation (RAG) systems must preserve the reliability and traceability of evidence as it moves from source documents into generated answers. This project introduces *Reliable Evidence Flow*, a system-level approach implemented in an integrated IBM Granite-based Retriever–Selector–Generator system. We evaluated the approach through module-level experiments, comparisons with RAG baselines, and ablations of the complete system. The results showed that Reliable Evidence Flow improved the preservation and grounded use of relevant evidence, limited the influence of misleading information, and produced clearer, more verifiable links between generated claims and their supporting sources. These findings highlight the complete evidence path as a central level for designing and evaluating reliability and traceability in enterprise RAG. The resulting system connects information discovery with evidence-grounded answer formation, keeping generated responses supported by evidence that remains identifiable from source to citation.

Keywords: Retrieval-Augmented Generation; Enterprise Question Answering; Evidence Grounding; Answer Attribution; Citation Verification.

Acknowledgements

We would first like to express our sincere gratitude to IBM US, particularly our industry mentor, Bharat Arora, for providing the opportunity, technical guidance, insightful feedback, and continued support throughout the development of this project. We are also grateful to our academic supervisor, Raphael Clifford, for his valuable advice, constructive feedback, and guidance throughout the project.

Author's Declaration

We declare that the work in this dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Taught Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, the work is the authors' own work. Work done in collaboration with, or with the assistance of, others is indicated as such. Any views expressed in the dissertation are those of the authors.

SIGNED: Jiayi You, Weikai Mao, Ming Wei, Zenan Wu,
Zhanyu Xu, Mengxiao Wu, Frida Chen

DATE: September 2026

Contents

List of Tables	vi
List of Figures	vii
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Formulation and Scope	2
1.3 Aim, Objectives and Contributions	2
2 Background	4
2.1 Technical Foundations and Challenges	4
2.2 Existing Approaches and Limitations	5
2.3 Research Positioning	5
3 Design and Implementation	7
3.1 Overall System Architecture	7
3.2 Core Module Design	8
3.2.1 Retriever Design	8
3.2.2 Selector Design	11
3.2.3 Generator Design	16
3.3 System Implementation	19
3.3.1 Module Development and Integration	19
3.3.2 Experiment Execution Environment	21
3.3.3 Efficient Implementation	22
3.3.4 Front-End Design and Visualisation	23
3.3.4.1 Main Features	23
3.3.4.2 Interface and Testing	24
3.3.5 System Evolution and Iterative Development	25
3.3.6 Team Coordination, Feedback and Delivery	27
3.3.7 Version Control and Development Traceability	29

4	Experimental Evaluation and Results	30
4.1	Evaluation Overview	30
4.2	Module-Level Evaluation and Results	31
4.2.1	Retriever Evaluation and Results	31
4.2.2	Selector Evaluation and Results	34
4.2.3	Generator Evaluation and Results	36
4.3	End-to-End Evaluation and Results	40
4.3.1	Experimental Setup	40
4.3.2	Main System Comparison	41
4.3.3	Module Ablation	42
5	Conclusion	45
5.1	Overall Conclusion	45
5.2	Critical Evaluation	45
5.3	Future Work	46
5.4	Contribution Statement	47
A	Technical Dependencies	48
	References	49

List of Tables

4.1	Overview of the experimental evaluation.	30
4.2	MRR and Recall@10 of eight Retriever configurations across three develop- ment benchmarks.	32
4.3	Paired MRR differences between Retriever configurations and their corre- sponding baselines.	32
4.4	Evidence-level results for the final Protect-Harm Selector under misleading- evidence stress tests.	35
4.5	Paired comparisons between Generator configurations on ASQA.	38
4.6	Paired comparisons between Generator configurations on QAMPARI. . . .	39
4.7	End-to-end evaluation metrics ($\uparrow$: higher is better; $\downarrow$: lower is better). . . .	40
4.8	Main system comparison: (a) KILT-NQ.	41
4.9	Main system comparison: (b) KILT-TriviaQA.	41
4.10	Main system comparison: (c) ALCE-ASQA.	41
4.11	Module ablation results: (a) KILT-NQ.	43
4.12	Module ablation results: (b) KILT-TriviaQA.	43
4.13	Module ablation results: (c) ALCE-ASQA.	43
A.1	Python dependencies	48

List of Figures

3.1	Overall architecture of the Granite-based RAG system.	7
3.2	Sparse-first hybrid Retriever.	10
3.3	RRF implementation with deterministic tie-breaking (<code>retriever/fusion.py</code>).	11
3.4	Selector Stage 1: corroboration-based evidence reranking.	12
3.5	Selector Stage 2: conflict-aware evidence filtering.	13
3.6	Selector Stage 3: three-class beam selection.	14
3.7	NLI-aware Protect-Harm Selector.	15
3.8	Generator Stage 2: draft generation and evidence verification.	17
3.9	Verify-and-Label Generator workflow.	18
3.10	Data contracts between RAG pipeline components.	20
3.11	Core execution sequence of <code>EvidenceRAGPipeline.run_with_trace</code>	20
3.12	Experiment workflow between the local machine and the BluePebble cluster.	21
3.13	Inverted-index construction and sparse query processing.	22
3.14	Evidence RAG prototype interface.	25
3.15	Project timeline.	26
3.16	Three-stage development of the RAG system.	27
3.17	Team coordination and review cycle.	28

1 Introduction

1.1 Context and Motivation

Enterprise organisations hold large volumes of structured, unstructured and multimodal information. This information is distributed across reports, policies, records, tables and scanned documents. A specific question may require only a small fraction of the collection. Finding that relevant information among large amounts of similar or irrelevant content is the needle-in-a-haystack problem addressed by this project.

Against this background, IBM asked us to investigate how Granite foundation models could improve the discovery of rare, relevant information in enterprise-scale data. The brief positioned semantic retrieval and retrieval-augmented generation (RAG) as the technical starting points for this investigation. In RAG, external information is retrieved and provided to the model as evidence for generating an answer, connecting information discovery to the quality of the resulting response. We therefore understood precision and recall as measures of retrieval performance, while explainability and trust addressed how to force the generated answer to align with the retrieved information. Through weekly meetings and Slack discussions, we refined this interpretation with the client, who provided four areas for further exploration: advanced prompting, hybrid retrieval, explainability, efficiency, and encouraged us to pursue a novel, research-led direction that could advance current research and potentially lead to publication.

Taken together, the brief and our discussions with the client connected enterprise information discovery with the reliability of generated answers. We therefore focused the project on the reliability and traceability of answer-supporting evidence from source documents to the final response.

1.2 Problem Formulation and Scope

The reliability of this evidence path begins with semantic retrieval, which identifies content related to a question. It may still miss evidence required for the answer. Passages that mention the same entities or topic can also appear highly relevant while describing a different event, time or relationship. Such passages may be retained during context construction despite this factual mismatch, obscuring or displacing required evidence. Evidence that reaches the generation stage may still be ignored, misused or incorrectly cited. The output of each stage determines what evidence is available to the next. In this project, reliability concerns whether required evidence remains available and is used correctly, while traceability concerns whether evidence supporting the final answer can be linked back to its source documents. The project therefore addresses the following research question:

How can a Granite-based RAG system maintain the reliability and traceability of answer-supporting evidence from source documents to the final response?

Accordingly, the project examines the complete path of answer-supporting evidence through retrieval, context construction and answer generation in a Granite-based RAG system. Its scope covers reliability within these stages and traceability between source documents and the final answer. The work is limited to a research prototype; production deployment, organisation-scale infrastructure and integration with live enterprise systems remain outside the project scope.

1.3 Aim, Objectives and Contributions

The overall aim is to improve evidence-grounded answering over large enterprise information collections by developing a Granite-based RAG system that maintains reliable and traceable supporting evidence. To pursue this aim, the project set three objectives:

1. To design and implement a Granite-based RAG system that retrieves information from enterprise-scale collections and produces answers linked to their source material.
2. To investigate and develop methods for reducing the loss of required evidence,

1 Introduction

controlling misleading context, and improving how evidence is used and attributed in generated answers.

3. To compare the proposed methods with appropriate baselines and determine how they affect the evidence available to the system and the quality of the final answer.

The project makes three contributions:

1. **Granite-based RAG system:** An integrated RAG system based on the IBM Granite model that can retrieve relevant information more accurately and generate answers which are faithful to the retrieved documents.
2. **Reliable Evidence Flow approach:** a system-level approach that connects evidence retrieval, context formation and claim grounding within a single answer-forming process.
3. **Cross-stage findings:** empirical findings on how evidence availability and context changes influence answer correctness and attribution.

2 Background

2.1 Technical Foundations and Challenges

Retrieval methods usually rank candidate passages according to their relevance to a query [1]. This ranking helps identify related information, but it assesses passages individually. Some questions require supporting facts from several documents [2]. A highly ranked passage may therefore contain only part of the evidence required for the answer. Retrieval depth and context length are also limited. Repeated or partly relevant passages may take the place of other necessary evidence. Retrieval must therefore be assessed through both individual passage relevance and the coverage of the retrieved set. The central question is whether the set makes enough answer-supporting evidence available for a complete answer.

After retrieval, context construction determines which passages are provided to the model and where they appear. Both selection and ordering can affect answer generation. Model performance can change when relevant information appears in different positions within a long context [3]. Irrelevant passages can also reduce the quality of the generated answer [4]. These findings show that context changes can make useful evidence easier to use, or remove evidence required for the answer as well. Evidence that remains in the context may still be overlooked or used incorrectly, such as being ignored by the model or attributed to a source that does not support the claim. Answer correctness and citation quality are therefore evaluated as separate dimensions [5]. Retrieval relevance, answer correctness and attribution must each be considered when evaluating the final answer.

2.2 Existing Approaches and Limitations

Research on retrieval first addresses how evidence enters the candidate set. Sparse retrieval relies on lexical overlap between a query and a passage. Dense retrieval uses semantic representations to recover relevant information expressed in different wording [6]. Hybrid retrieval combines lexical and semantic signals because they often recover complementary evidence [7]. Multi-hop retrieval supports questions that require connected evidence from several passages or documents [2]. These methods improve matching and evidence coverage in different ways. A remaining gap is how to recover sufficiently complete answer-supporting evidence when the required support appears in lexical, semantic and relational forms.

Once candidate passages have been retrieved, context construction determines which of them remain available for generation. Reranking methods prioritise the strongest candidates, while set-selection methods consider passages jointly [8, 9]. Reliability-aware methods identify conflicts between passages [10]. Risk-control methods calibrate selection against a defined loss [11]. These approaches can reduce irrelevant, conflicting or misleading material. They may also remove evidence required for the answer. Their value therefore depends on the evidence that remains available and the quality of the final answer.

Even a well-constructed context does not ensure that its evidence will be used correctly. Evidence-aware generation incorporates passage support into the generation process [12]. Benchmarks for citation-aware methods connect generated claims to source passages and evaluate citation correctness and completeness [5]. Independent checkers such as MiniCheck verify generated claims against grounding documents [13]. These methods improve how evidence is used and attributed in generated answers. However, answer correctness and attribution remain distinct. A correct answer may contain unsupported citations, while well-supported citations do not ensure a complete or correct answer.

2.3 Research Positioning

The approaches reviewed above improve different parts of the answer-forming process. Retrieval methods seek to make required evidence available. Context construction changes which evidence remains available for generation. Generation and verification methods

2 Background

influence how that evidence is used and attributed. Each approach provides useful local improvements, but a broader gap remains in understanding how these changes are connected. This gap concerns how evidence availability and context changes influence answer correctness and attribution across the answer-forming process. Reliable Evidence Flow provides a system-level perspective for examining these relationships. It treats answer-supporting evidence as the common research object across retrieval, context construction and answer generation. Within this perspective, the methods used in the project are coordinated interventions on the same evidence. Their value is assessed through both their direct effects and the quality of the final answer. The following chapters explain how this perspective informed the system design, method selection and evaluation strategy.

3 Design and Implementation

3.1 Overall System Architecture

The preceding chapter showed that reliable evidence-grounded answering depends on three connected stages: retrieving sufficient answer-supporting evidence, constructing the context supplied to the model, and using and attributing that evidence in the final answer. Based on this analysis, we organised the Granite-based prototype as a Retriever–Selector–Generator pipeline. Each module addresses one stage of the answer-forming process, allowing the Reliable Evidence Flow perspective to guide the overall system design. Figure 3.1 presents this end-to-end architecture and the flow of evidence between the three modules.

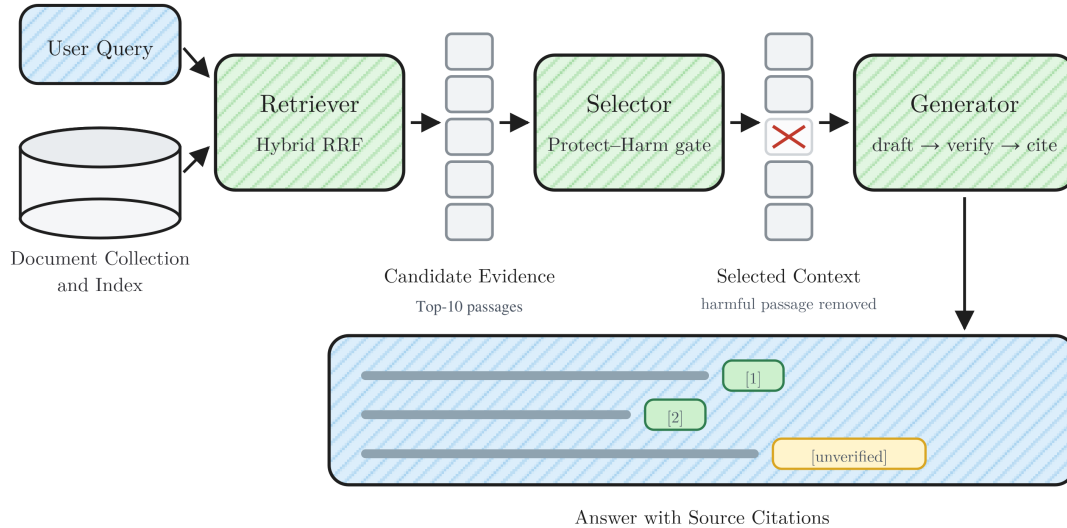


Figure 3.1: Overall architecture of the Granite-based RAG system.

The system contains three modules:

1. **Retriever:** searches the indexed document collection and returns a ranked set of

3 Design and Implementation

candidate evidence relevant to the user query.

2. **Selector:** constructs the generation context by reducing potentially distracting or misleading information while preserving evidence required for the answer.
3. **Generator:** uses the selected context to produce an answer, links its statements to the supporting source material through citations, and labels all other claims that fail verification.

Together, the three modules form a continuous pipeline from evidence retrieval to source-grounded answer generation. The following section presents the method design of each module in turn.

3.2 Core Module Design

3.2.1 Retriever Design

The Retriever searches the indexed document collection and returns a ranked set of passages for a user query. Its goal is to retrieve as complete a set of relevant evidence as possible within a fixed number of results. Relevant information may share key terms with the query, be expressed in different wording, or be distributed across several documents.

Design Evolution

Stage 1: Establishing a Keyword-Based Baseline

We began with BM25, which ranks passages by the words they share with the query and gives greater weight to more distinctive matches. We then tested a commonly used BM25 configuration with stopwords filtering. Frequent but uninformative words such as “the” and “of” were ignored, allowing more meaningful terms to have greater influence on the ranking. The revised configuration did not clearly increase the amount of relevant evidence retrieved. Keyword matching works best when the query and the passage use the same terms. It may miss relevant evidence when the wording changes. For example, a query may use “wrote”, while a relevant passage uses “was authored by”. Such differences in wording

3 Design and Implementation

cannot be captured through direct word overlap alone. We therefore retained BM25 as the keyword-based foundation and introduced semantic retrieval in the next stage.

Stage 2: Combining Keyword and Semantic Retrieval

To find relevant passages that used different wording from the query, we added a Granite bi-encoder to BM25. BM25 ranked the candidate passages using shared keywords. The Granite bi-encoder converted the query and the same passages into numerical representations, then ranked the passages according to their similarity in meaning. We tested two methods for combining the keyword and semantic rankings. Reciprocal rank fusion (RRF) assigned each passage a value based on its position in both lists, with higher positions making a larger contribution to the final ranking. Score-level fusion instead placed the BM25 and semantic scores within the same range and combined them using a weighted sum. RRF offered a more consistent way to combine the two signals because it used rank positions and required no adjustable weighting between the methods. We therefore selected RRF as the fusion method. Even after fusion, both rankings still began with the original query. A short query may provide too little context, while a complex query may contain several information needs that are difficult to search for together. To address both cases, we next tested whether expanding or dividing the query could make the required evidence easier to find.

Stage 3: Testing Changes to the Query

We first explored whether adding context to short queries could improve retrieval. Query2Doc expanded each query into a short passage written in the style of a source document, and BM25 used the additional wording to find keyword matches. HyDE instead generated a hypothetical answer passage and used its meaning to guide semantic retrieval. Neither method improved retrieval consistently across different query types, so query expansion was not included in the default design.

We also tested query decomposition for complex queries that required several connected facts. Each query was divided into simpler sub-questions and searched separately, after which RRF combined the results. Decomposition mainly reordered existing passages and sometimes favoured broadly related passages over evidence supporting a specific fact. It did not improve retrieval consistently. We therefore retained the original query and excluded

3 Design and Implementation

decomposition from the default design. Detailed comparisons across all three design stages are reported in Section 4.2.1.

Final Design: Two-Stage Hybrid Retriever

The final Retriever keeps the original query unchanged and follows a sparse-first hybrid process. BM25 first searches the complete indexed collection and selects the 1,000 passages with the strongest keyword matches. These passages define the candidate pool for all remaining steps. The Granite bi-encoder compares the meaning of the original query with each passage in the pool and produces a semantic ranking. RRF combines the positions of each passage in the BM25 and semantic rankings, giving greater influence to passages placed near the top of either list. The fusion uses $k = 60$, and the ten highest-ranked passages form the **CandidateSet**. Across the three development benchmarks, this design was the only tested addition that improved evidence ranking in every setting. We therefore adopted it as the final Retriever configuration. The resulting sparse-first hybrid process is summarised in Figure 3.2.

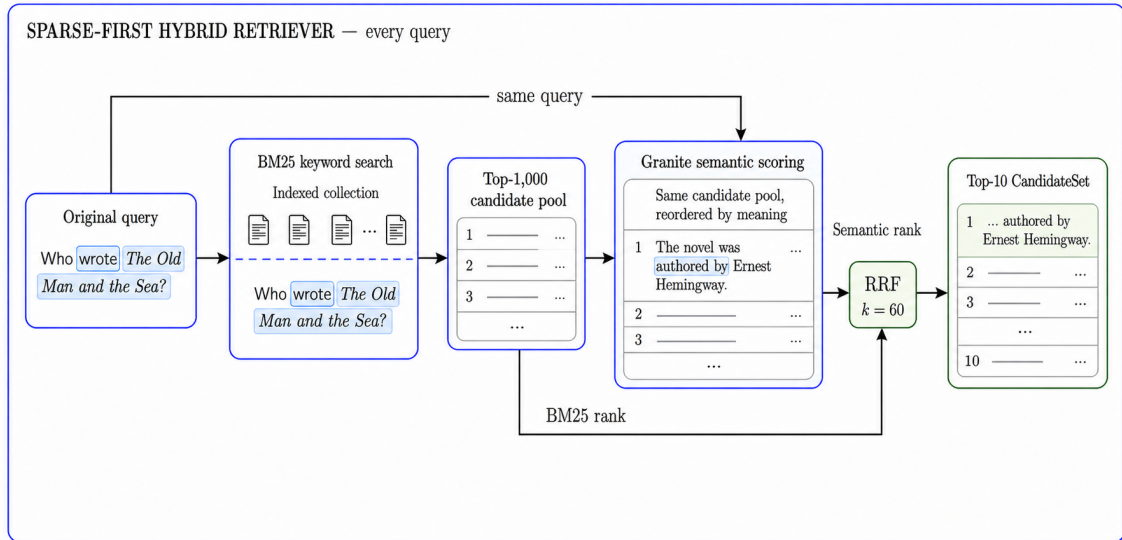


Figure 3.2: Sparse-first hybrid Retriever.

Implementation.

We bind each saved index to the Retriever implementation, version, parameters and corpus snapshot that created it. When the runner loads an index, it checks this information

3 Design and Implementation

against the active configuration before processing queries. We implement rank fusion through one shared function used by both the hybrid and decomposing retrievers. The configuration layer checks that rank-level fusion receives at least one ranking and score-level fusion receives exactly two, and rejects unknown fusion names before execution. The fusion function calculates the scores without changing its input `CandidateSet` objects and returns a new ranked set. It resolves tied scores using an optional secondary score and then the evidence identifier. Figure 3.3 shows the central RRF implementation and its deterministic tie-breaking rule.

```
for result, weight in zip(results, _arm_weights(results, weights), strict=True):
    for candidate in result.candidates:
        scored[candidate.evidence_id] = scored.get(candidate.evidence_id, 0.0) \
            + weight / (k + candidate.retrieval_rank)
        representative.setdefault(candidate.evidence_id, candidate)
return _ranked(scored, representative, query_id=query_id, top_k=top_k)

# _ranked: fused score, then optional secondary score, then evidence_id
ordered = sorted(
    scored.items(),
    key=lambda item: (-item[1], -secondary.get(item[0], 0.0), item[0]),
)
```

Figure 3.3: RRF implementation with deterministic tie-breaking (`retriever/fusion.py`).

3.2.2 Selector Design

The Selector operates between the Retriever and the Generator. It receives the ranked candidate evidence returned by the Retriever and decides which passages enter the generation context. It does not retrieve new evidence or replace the original ranking. Its task is to reduce information that may mislead the Generator while preserving the evidence needed to answer the query. This includes evidence distributed across several passages. The three development stages below show how the design moved from reranking candidates to making this preservation-removal decision directly.

3 Design and Implementation

Design Evolution

Stage 1: Corroboration-Based Reranking

The first design explored corroboration: whether the same answer was supported by several independent documents. For each query, the method extracted a candidate answer from every retrieved passage. It then normalised small differences in wording and grouped passages that expressed the same answer. The number of distinct documents in each group became a corroboration score. This score was combined with the Retriever’s original relevance score to produce a new ranking.

Tests on the NQ/DPR development benchmark showed that this method helped some target evidence remain within the Top-10 candidates. However, it did not consistently move the first relevant passage higher in the ranking. More importantly, repeated information was not necessarily correct. Corroboration could measure agreement, but it could not determine which passage should be removed when the retrieved evidence contained competing claims. We therefore shifted the focus from agreement to conflict. Figure 3.4 summarises this corroboration-based reranking stage.

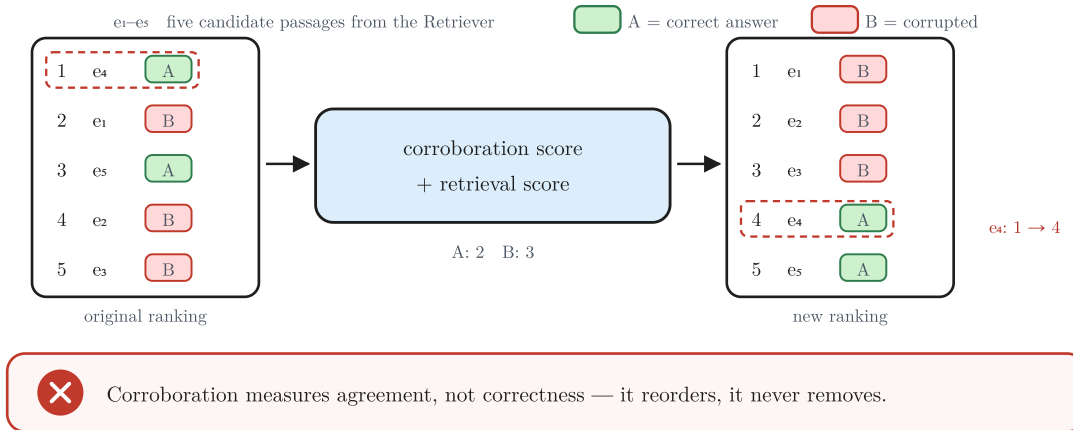


Figure 3.4: Selector Stage 1: corroboration-based evidence reranking.

Stage 2: Conflict-Aware Evidence Filtering

The second design examined whether retrieved passages made incompatible claims. Passages expressing the same normalised answer were placed in one group. Support from distinct source documents was then counted for each group. When two groups conflicted, the

3 Design and Implementation

method could remove a passage associated with the less-supported answer.

This initial design relied heavily on answer strings, so it could miss conflicts expressed in different language. We therefore introduced Natural Language Inference (NLI), which identifies whether one piece of text supports, contradicts or is unrelated to another. NLI allowed the method to compare meaning rather than exact wording. Evaluation showed that conflict-aware filtering removed more harmful evidence, but it also removed some evidence required for the answer. It also produced false conflicts when the relationship between passages was uncertain. Detecting disagreement was therefore useful, but disagreement alone did not show whether a passage should be retained. Figure 3.5 illustrates how this second stage used conflict detection to filter evidence.

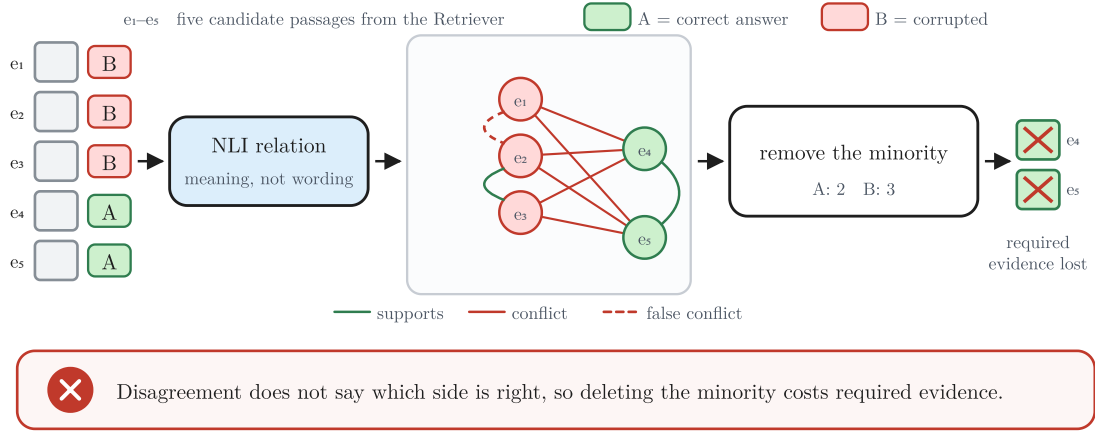


Figure 3.5: Selector Stage 2: conflict-aware evidence filtering.

Stage 3: Three-Class Beam Selection

The third design assessed the role of each passage more directly. It classified candidates as required, harmful or irrelevant. Required passages contained information needed for the answer. Harmful passages could mislead the Generator, while irrelevant passages added no useful support. The method then used beam search to maintain several promising partial evidence sets and expand them step by step. This allowed combinations of passages to be considered when an answer required evidence from more than one source.

We implemented the method and tested it on the NIAH and 2WikiMultiHopQA development sets. It removed substantially more harmful evidence than the earlier designs. However, it

3 Design and Implementation

also caused a clear loss of required evidence. The three classes forced every passage into one role, even when a passage contained both useful and potentially misleading information. Preservation and removal had become competing outcomes of the same prediction. This result led us to treat them as two separate decisions in the final design. The three-class beam-selection process is summarised in Figure 3.6.

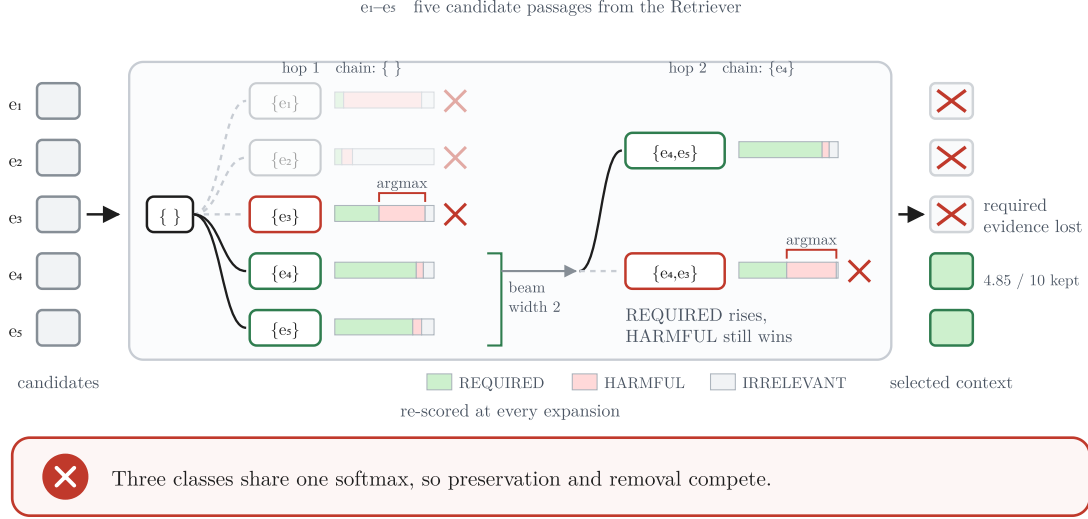


Figure 3.6: Selector Stage 3: three-class beam selection.

Final Design: NLI-Aware Protect-Harm Selector

The results of the three-class beam selector made the remaining limitation clear. Treating evidence retention and removal as opposite outcomes of a single classification encouraged aggressive filtering when the model was uncertain. We therefore designed the final Selector to assess these two decisions separately. Instead of assigning each passage to one fixed role, the method estimates both the value of preserving the passage and the risk of allowing it to enter the generation context.

For each query q and candidate passage e_i , the Selector processes the query-passage pair using a DeBERTa model pretrained for Natural Language Inference (NLI), which determines whether one text entails, contradicts, or remains neutral to another. The shared NLI representation is passed to two independent prediction heads. The first produces a protect score p_i , indicating how strongly the passage should be preserved as answer-supporting evidence. The second produces a harm score h_i , indicating the likelihood that the passage

3 Design and Implementation

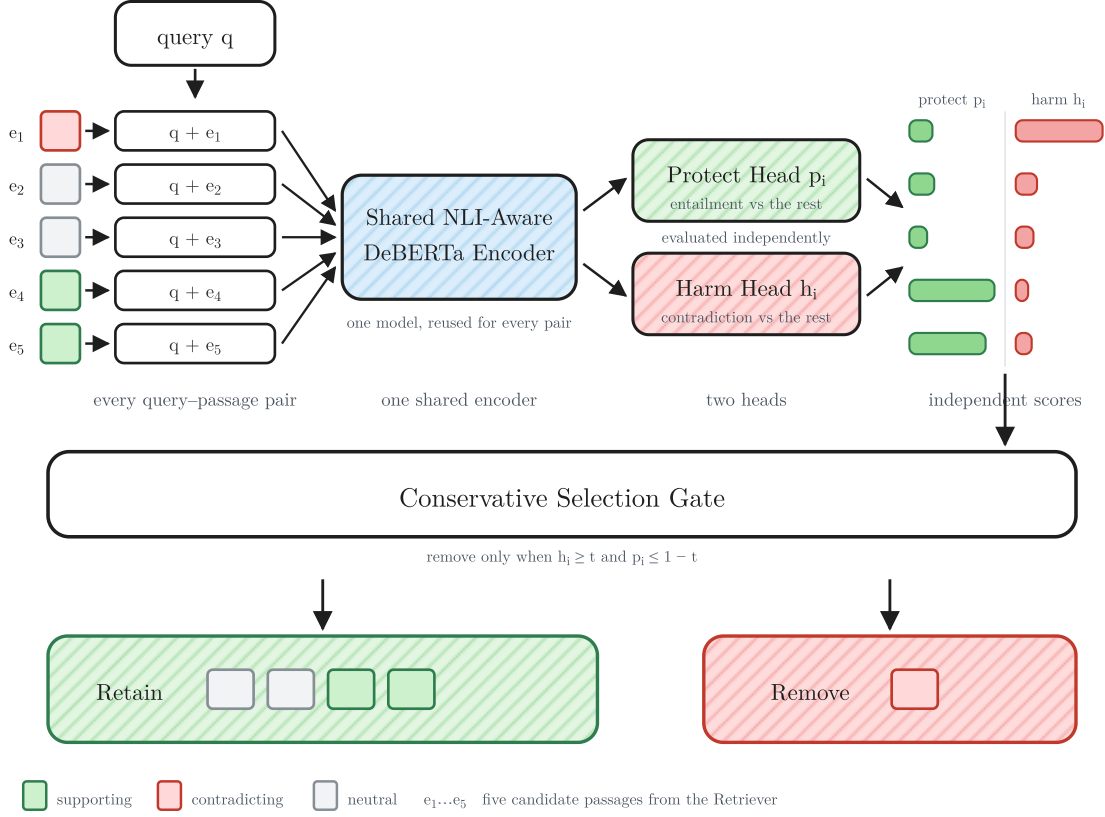


Figure 3.7: NLI-aware Protect-Harm Selector.

contains contradictory or misleading information. The protect head gives a high score to passages that support the answer, while the harm head gives a high score to passages that contradict it. Neutral passages receive low scores from both heads. Because the two scores are estimated independently, a passage is not forced into a single mutually exclusive category. Figure 3.7 shows the shared NLI representation and the two independent protect and harm heads.

At selection time, deletion is triggered by a joint high-harm, low-protect condition. A candidate passage is removed when its harm score reaches or exceeds the threshold t and its protect score remains at or below the corresponding protection boundary, expressed as $h_i \geq t$ and $p_i \leq 1 - t$. This rule targets passages that are likely to mislead the Generator while providing little useful support for the answer. The remaining passages are then passed to the Generator as the evidence context for answer generation.

We evaluated the dual-score design at the Selector level using two independently trained

3 Design and Implementation

seeds. It reduced harmful evidence by 12.95% and 13.10%, with deletion precision of 75.44% and 80.56%, respectively. Importantly, neither run caused a measurable loss in required-evidence recall. Both runs also preserved multi-hop evidence chains by retaining the related passages needed to combine information from multiple sources. We therefore selected the NLI-aware Protect–Harm design as the final Selector configuration. The complete experimental settings, comparative results, and end-to-end effects on answer generation are presented in Chapter 4.

3.2.3 Generator Design

The Generator uses the evidence retained by the Selector to produce the final response. Its goal is to answer the query faithfully from this context while maintaining a clear connection between the generated claims and their supporting source material. Within Reliable Evidence Flow, the Generator therefore carries the preserved evidence into the final answer, supporting both answer reliability and source-level traceability.

Stage 1: Generation-Time Citation

To connect the final answer to the evidence retained by the Selector, we first asked the Generator to produce the answer and its citations in a single step. Each selected passage was assigned a source identifier, such as [1] or [2], which the model placed after the factual statement it was intended to support. This allowed readers to trace each citation back to the evidence supplied to the Generator.

However, because the statement and its citation were generated together, the citation reflected the model’s intended attribution rather than an independent confirmation of support. Evaluation showed that some citations pointed to related but insufficient evidence, while the supporting passage sometimes appeared elsewhere in the selected context. Generation-time citation therefore made the evidence path visible, but it asserted support rather than demonstrating that the attribution was valid.

Stage 2: Verifying Generated Claims against Evidence

At this stage, we separated answer production from the assessment of evidential support. A draft generator used the query and selected evidence to produce an initial answer with

3 Design and Implementation

provisional citations. A claim splitter then divided this draft into atomic claims, meaning factual statements that could be checked independently. A separate evidence verifier compared each claim with the selected passages and returned a support decision rather than generating new answer content. Figure 3.8 illustrates this separation between draft generation and evidence verification.

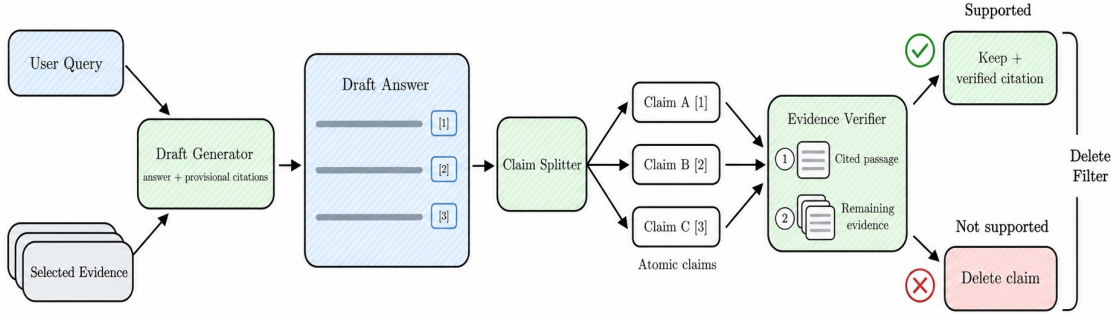


Figure 3.8: Generator Stage 2: draft generation and evidence verification.

For each claim, the verifier first examined the passage named by the provisional citation and then the remaining selected passages. If it identified supporting evidence, the claim remained in the answer with the verified source. If no support was identified, the claim was removed. We refer to this deletion-based configuration as the Delete Filter. It made citation validity a separate decision from answer generation.

The Delete Filter strengthened the evidential basis of the remaining content, but due to limitations of the verifier’s performance, it could not recognise every valid claim–evidence relationship. Under the deletion policy, each missed relationship removed potentially correct content without leaving any indication in the final response. Evaluation therefore showed a clear trade-off: citation quality improved, while answer coverage and correctness significantly decreased.

3 Design and Implementation

Final Design: Verify-and-Label Generator

We therefore retained the separation between drafting and verification introduced in Stage 2, but replaced deletion with a Verify-and-Label policy. Citations produced during drafting remain provisional until their support has been checked. The final answer can consequently distinguish between claims linked to verified evidence and claims whose support has not been established from the selected context.

The draft generator produces an initial response from the query and selected evidence. Its prompt encourages one independently checkable fact per sentence and asks the model to attach a provisional source identifier. A claim splitter then separates the response into atomic claims while retaining their positions in the original draft. For each claim, the evidence verifier first checks the passage named by the provisional citation and then examines the remaining selected passages. Searching the complete selected context allows the system to correct the original attribution when another passage provides the actual support. The resulting claim-level decisions are then assembled back into the final response. The complete Verify-and-Label workflow is shown in Figure 3.9.

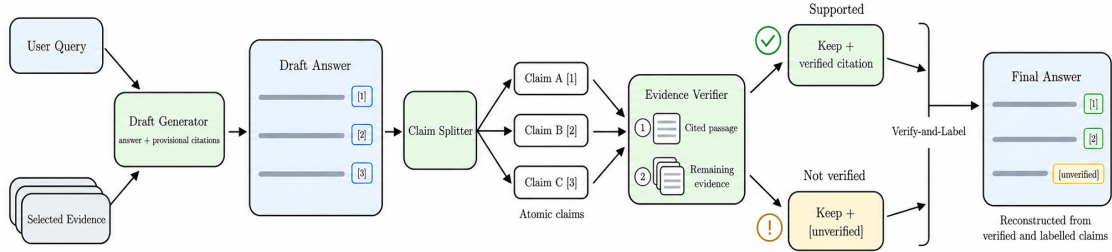


Figure 3.9: Verify-and-Label Generator workflow.

The final configuration uses TRUE [14] to determine whether a passage supports a claim. Among the tested verifiers, TRUE produced the most precise support decisions on labelled claim-evidence pairs. Claims with identified support receive citations, while claims without identified support remain visible as `[unverified]`. The output rule requires every claim without a verified citation to carry this label, keeping the evidential status of the response explicit.

3 Design and Implementation

Module-level evaluation showed that Verify-and-Label improved citation precision and recall compared to Generation-Time Citation, while preserving substantially more answer coverage and correctness than the Delete Filter. The complete experimental setup, evaluation metrics and comparative results are presented in Section 4.2.3.

3.3 System Implementation

3.3.1 Module Development and Integration

We implemented the RAG system in Python 3.11 as the `evidence_rag` package. The Retriever, Selector and Generator were maintained as separate components, while their shared code was organised into `contracts`, `pipeline` and `composition.py`. The contracts defined the inputs and outputs of each component. The pipeline coordinated their execution, and `composition.py` assembled the module versions specified by an experiment configuration. As new methods were developed, they could be added behind the same interfaces and evaluated through the same execution path. This allowed the three modules to evolve independently before their selected versions were integrated into the complete system.

Connecting these components required a clear definition of the data exchanged at each boundary. We represented this data using immutable Pydantic models. The Retriever returns a `CandidateSet` containing the retrieved passages, their source information, retrieval scores and ranks. The Selector receives these candidates and returns a `SelectionResult` containing the identifiers of the evidence it has selected. The Pipeline then uses these identifiers to recover the corresponding passages from the original candidate set and constructs a `SelectedEvidenceSet`. The evidence passed to the Generator therefore retains the text, source and retrieval information produced by the Retriever. A lightweight `QueryChecklist` is also provided to record the focus and main requirements of the query. The Generator uses these inputs to produce a `GenerationResult` containing the answer and its cited evidence identifiers. The resulting data contracts and information flow between components are summarised in Figure 3.10.

These data contracts are applied by `EvidenceRAGPipeline`, which provides a common

3 Design and Implementation

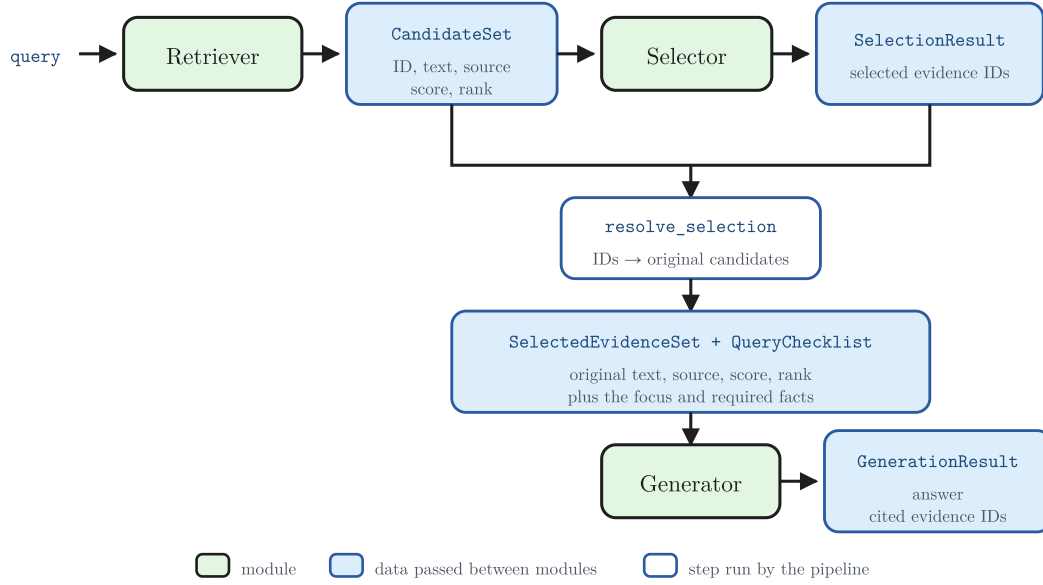


Figure 3.10: Data contracts between RAG pipeline components.

execution path for different module combinations. For each query, the Pipeline first creates the `QueryChecklist`. It then runs retrieval, selection and generation in sequence. The central execution sequence is shown in Figure 3.11.

```

checklist = self.query_analyzer.analyze(query)
candidates = self.retriever.retrieve(query, top_k)
selection = self.selector.select(query, candidates, max_selected)
selected = resolve_selection(candidates, selection)
result = self.generator.generate(query, checklist, selected)
validate_generation(selected, result)

```

Figure 3.11: Core execution sequence of `EvidenceRAGPipeline.run_with_trace`.

After the Selector returns its decision, `resolve_selection` maps the selected identifiers back to the original evidence objects. The resulting context is then passed to the Generator. At each stage, the Pipeline checks that the query identifier remains consistent. It also checks that every selected identifier belongs to the candidate set and that every citation in the generated answer refers to selected evidence. The intermediate outputs and final answer are collected in a `PipelineRun`, providing a complete record of how the query moved through the system. The same execution record is used in the integration tests and

3 Design and Implementation

the evaluation presented in Chapter 4.

3.3.2 Experiment Execution Environment

We used the BluePebble high-performance computing (HPC) facility for experiments that required GPU resources. Code development, version control and analysis of the completed retrieval outputs, including the calculation of information-retrieval metrics, remained on a local machine. Code and experiment configurations were transferred through the shared GitHub repository. We pushed the selected version from the local machine and pulled it onto BluePebble’s login node. Model weights were downloaded from Hugging Face and stored in a shared cache on the cluster.

From the login node, we submitted each experiment with `sbatch`, and Slurm assigned the job to a GPU node. During execution, the job read its inputs from and wrote its outputs to `/user/work`. After the run finished, we copied the resulting files back to the local machine for analysis and reporting. This end-to-end experiment workflow is illustrated in Figure 3.12.

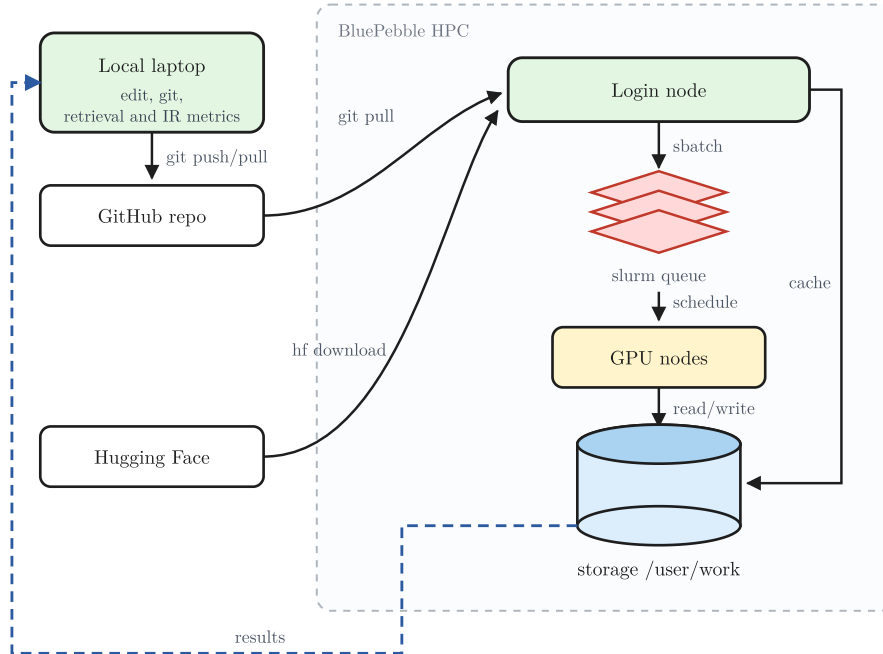


Figure 3.12: Experiment workflow between the local machine and the BluePebble cluster.

3 Design and Implementation

3.3.3 Efficient Implementation

After integrating the three modules into a common pipeline, we turned to the cost of running retrieval for every user query. When a question enters the system, the sparse stage of the two-stage Retriever first ranks the complete corpus according to the query terms shared with each chunk, then passes the top 1,000 candidates to the dense stage for semantic re-ranking. In the initial implementation, producing this candidate pool required a BM25 score to be calculated (a relevance score showing how well the chunk matches the query) for every chunk. Each query therefore required every chunk in the corpus to be processed. Query latency grew almost in proportion to corpus size: a 10.33-fold increase in the number of chunks resulted in a 10.47-fold increase in latency. We therefore focused our implementation optimisation on the sparse scoring stage. Figure 3.13 summarises the resulting index-construction and sparse query-processing workflow.

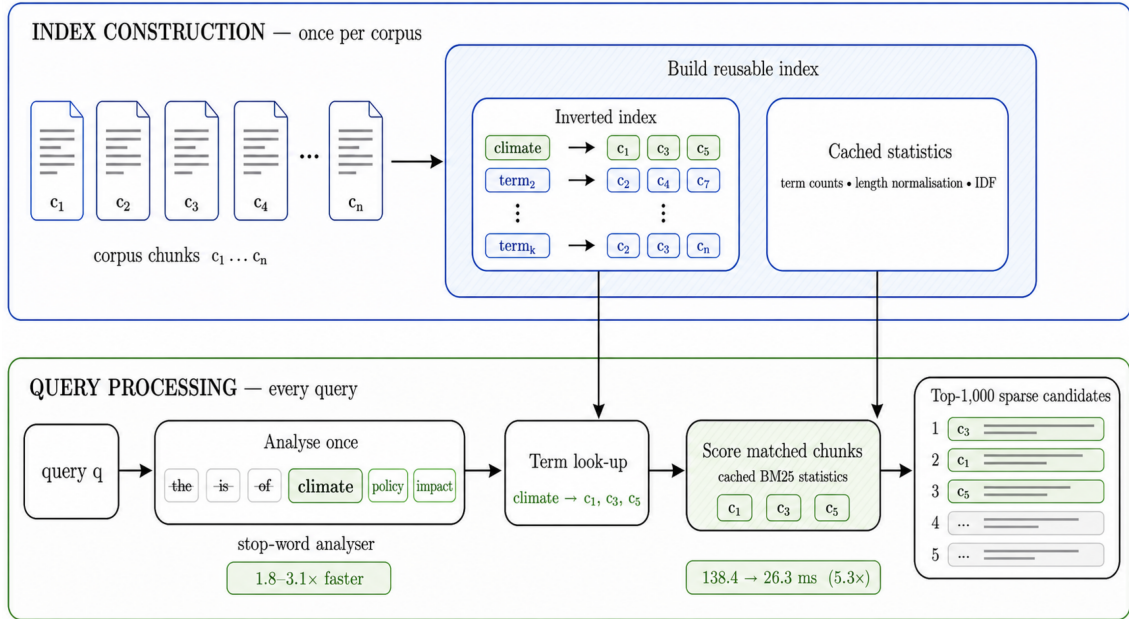


Figure 3.13: Inverted-index construction and sparse query processing.

We first addressed the calculations being repeated inside the scoring loop. The number of times a term appears in a chunk and the adjustment made for chunk length depend only on the corpus, while the analysed query remains fixed during a single retrieval. We therefore calculated the corpus-dependent values when building the index, analysed each query once, and reused both sets of values throughout scoring. For a query over 8,778 chunks, this

3 Design and Implementation

reduced latency from 138.4 to 26.3 ms, corresponding to a $5.27\text{--}5.40\times$ speed-up while producing identical scores and rankings.

Although this made each scoring operation cheaper, the Retriever still visited every chunk in the corpus. We therefore replaced the complete scan with an inverted index, which records the chunks containing each term in much the same way that the index at the back of a book directs a reader to relevant pages. A query can then follow the entries for its terms rather than search through the collection from beginning to end. Common words such as “the” and “of” occur in many chunks but provide little information about which chunks are relevant. Their index entries are therefore long and costly to process. We therefore used a stop-word analyser to remove these words before retrieval. On real text, this made retrieval $1.8\text{--}3.1\times$ faster. Together, the two optimisations reduce both the work required to calculate each score and the amount of the corpus that must be visited. The resulting ranked evidence is returned through the existing `CandidateSet`, which provides the identifiers, scores and ranks used by the Selector and Generator.

3.3.4 Front-End Design and Visualisation

The research prototype was implemented as a locally operated full-stack application. The backend uses Python, FastAPI, and Uvicorn, while the frontend uses Next.js, React, TypeScript, and Tailwind CSS. Where local inference is required, PyTorch, Hugging Face Transformers, and Safetensors are used to load the models and trained weights. Conversations and their associated evidence are stored locally in the browser, and the lightweight demonstration does not depend on an external inference API.

3.3.4.1 Main Features

The system was positioned as a research prototype rather than a production application. It demonstrates the final Retriever, Selector, and Generator configuration through a unified end-to-end workflow, integrating questions, retrieved evidence, Selector results, answers, and citations within one interface.

The evidence presentation was inspired by Perplexity’s source-oriented question-answering

3 Design and Implementation

interface. Answers are presented alongside retrieved evidence and citations, enabling users to inspect their supporting sources. For the research objectives of Evidence RAG, the interface additionally shows evidence-level Selector outputs, including associated scores, retention or removal decisions, and decision reasons. The design therefore adopts Perplexity’s emphasis on source transparency and citation traceability while extending it for pipeline analysis.

User Mode and Developer Mode were considered during the early design stage. The former would present only answers, citations, and evidence, while the latter would expose technical information such as Retriever outputs, Selector scores, module status, and model settings. Following feedback from the IBM client, the mode switch and user-configurable model settings were removed in favour of a unified research-demonstration interface. This reduced interface complexity, ensured that every demonstration represented the final experimental configuration, and retained the answer, cited evidence, and key Selector decisions.

The lightweight version uses reference fixture data by default, containing representative questions, retrieved passages, Selector decisions, citations, and reference answers. This supports reproducible validation of the complete interaction and visualisation flow without loading every large model locally. When the required model files are available, the relevant modules can also perform inference using local snapshots and trained checkpoints.

The application supports question answering, local document upload, evidence citations, Selector decision visualisation, and local conversation history. Implementation work also addressed unnecessary pipeline reinitialisation after document uploads, loss of incrementally presented content, and inconsistent synchronisation between the active conversation and local history.

3.3.4.2 Interface and Testing

The interface uses a three-column layout. The left panel manages conversation creation, switching, and deletion. The central panel presents the question, pipeline status, answer, and citations. The right panel displays the retrieved evidence and its Selector scores, filtering decisions, and decision reasons. This layout allows general users to focus on the answer and citations while enabling researchers to inspect the evidence-selection process without switching interface modes.

3 Design and Implementation

Reference fixture data was used to test question submission, status updates, incremental answer presentation, evidence rendering, citation mapping, Selector decisions, and conversation management. Fixed inputs and outputs enable deterministic and reproducible validation of the full-stack integration without real-time inference from every large model. These tests validate interface behaviour, data transfer, and result visualisation; model performance is reported separately through the offline evaluations in the experimental chapters. Figure 3.14 shows the resulting three-column interface and its evidence display.

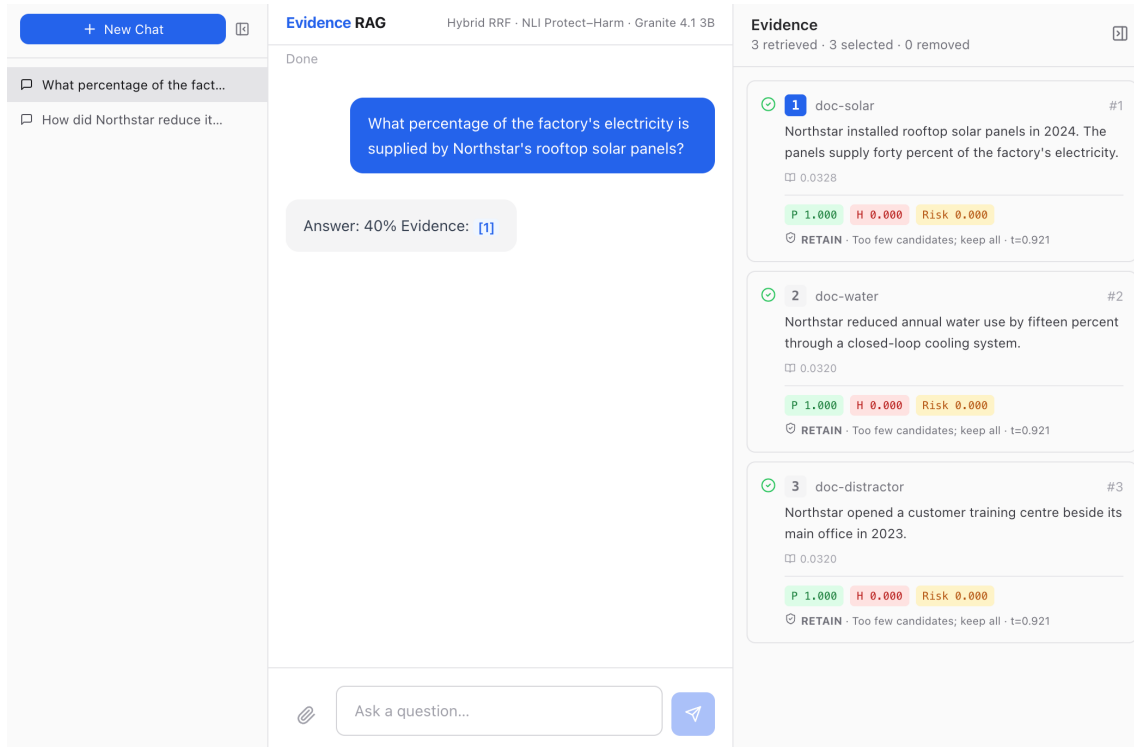


Figure 3.14: Evidence RAG prototype interface.

3.3.5 System Evolution and Iterative Development

Figure 3.15 presents the project timeline across the three development stages described below.

Stage 1: Establishing the Baseline RAG System (1 June–13 July)

We first built a basic RAG pipeline covering document processing, retrieval and answer generation. This version served as a shared baseline for early experiments and helped

3 Design and Implementation

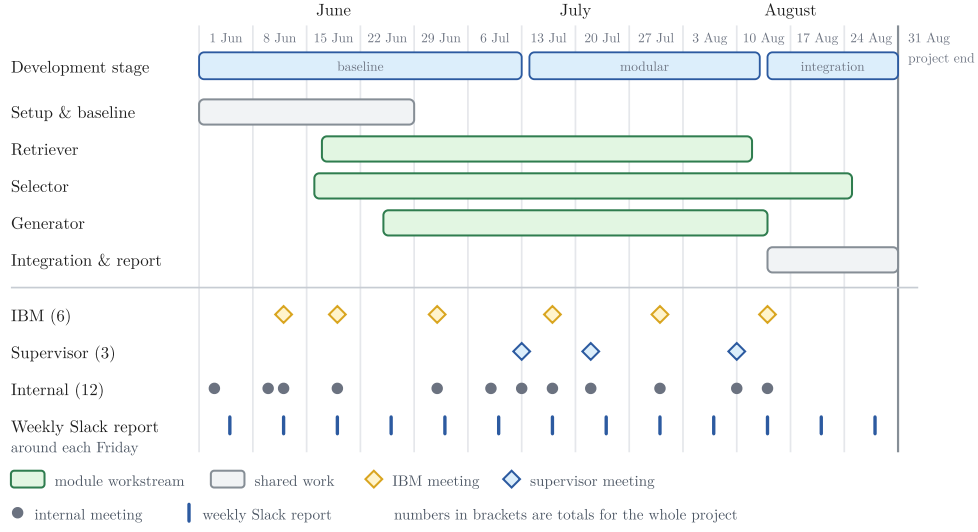


Figure 3.15: Project timeline.

clarify the main technical tasks. The implementation initially followed the end-to-end flow of the system, without stable boundaries between its main components. As more methods were introduced, changes in one part of the pipeline began to affect several others, making experimental comparisons harder to interpret. This experience led us to archive the baseline on 13 July and reorganise the codebase around a modular architecture.

Stage 2: Modular Development (14 July–13 August)

The new structure separated the Retriever, Selector and Generator through shared data contracts. Component owners could compare methods within their modules without rewriting the rest of the pipeline. Experiment plans, configurations and results were stored alongside the corresponding implementation.

Internal reviews covered evidence formats, query information and the data required by the Generator. We also compared unsuccessful approaches and the latest experimental results during these discussions. By mid-August, the module-level experiments had produced candidate Retriever, Selector and Generator versions for integration.

Stage 3: System Integration and Finalisation (14–31 August)

We connected the selected modules through the common pipeline described in Section 3.3. Integration exposed differences in configuration, evidence representation and the handling

3 Design and Implementation

of evidence identifiers between selection and generation. Shared smoke tests and integration tests were introduced as these issues were resolved. End-to-end runs then examined the behaviour of the combined system and informed the final configuration. The associated code, settings and output artefacts were retained for the evaluation in Chapter 4. The progression from the baseline to the integrated system is summarised in Figure 3.16.

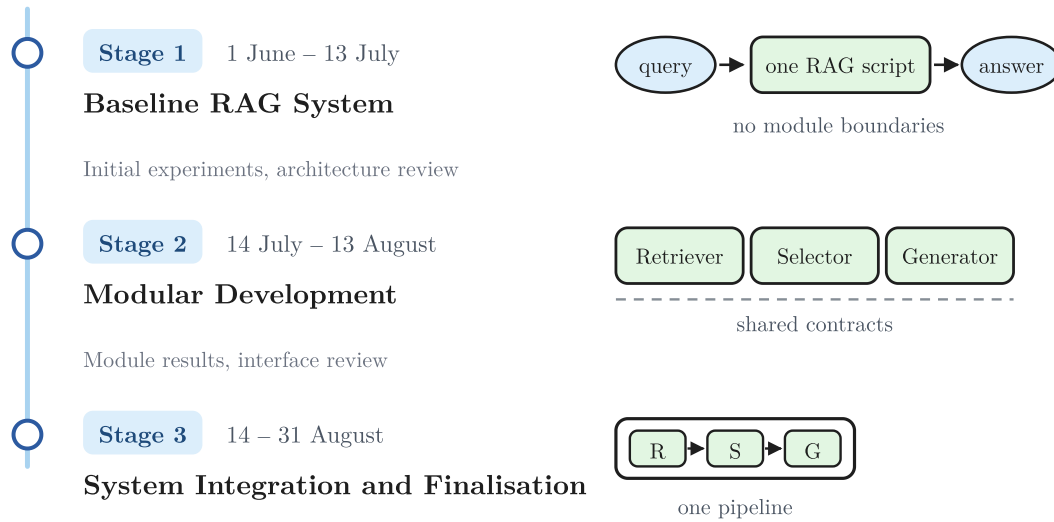


Figure 3.16: Three-stage development of the RAG system.

3.3.6 Team Coordination, Feedback and Delivery

We organised module development around the Retriever, Selector and Generator, with two members working on each module and one member developing the demonstration interface. At the beginning of each iteration, the module groups worked from the current experiment plan and the issues identified in the previous review. Candidate methods were implemented and evaluated against the criteria used by each module, with configurations and results recorded as the work progressed.

At the end of a development period, each module group prepared a short summary of its research focus, method design, experimental results, current problems and proposed next steps. These summaries formed the basis of an internal technical review. Results from the three modules were considered together, interface requirements were checked, and issues affecting more than one component were discussed. The main findings and decisions were

3 Design and Implementation

then consolidated into a single presentation or written progress report for IBM and the academic supervisor.

We maintained regular contact with IBM through online meetings and progress reports shared via Slack. Consolidated project updates were also presented in meetings with the academic supervisor. These discussions covered the overall research direction, the method designs for each module, experimental settings and evaluation criteria, interim results and identified limitations, and proposed adjustments to methods and development priorities for the next iteration.

Feedback from each external review was discussed internally and translated into changes to the experiment plans, task priorities and work assigned for the next iteration. Development then continued from the revised plan, producing a new set of results for the following review. Once the final designs of the three modules had been selected, the same cycle continued with pipeline integration and end-to-end experimental design. Results from smoke tests and full-pipeline runs were reviewed in the same way until the final system configuration was ready for evaluation. Figure 3.17 summarises this recurring cycle of development, internal review, external feedback and revision.

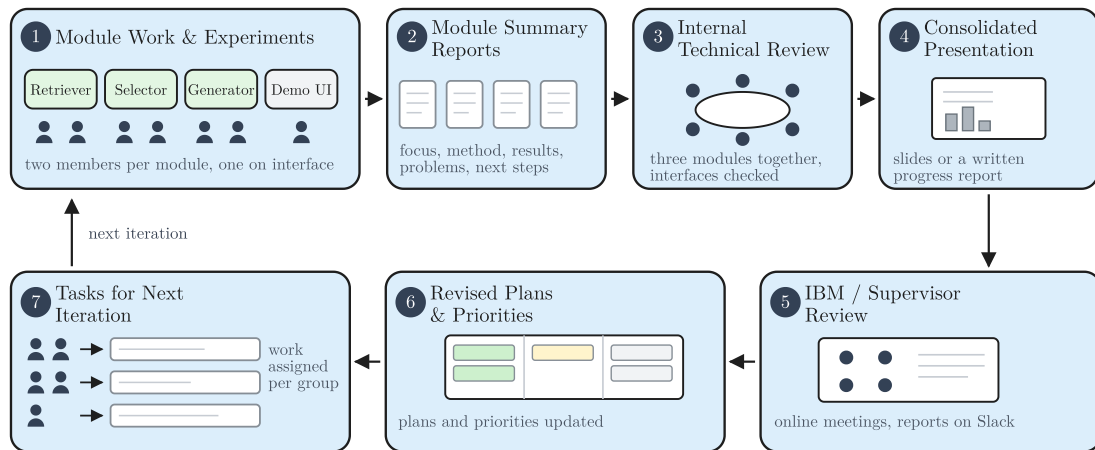


Figure 3.17: Team coordination and review cycle.

3.3.7 Version Control and Development Traceability

We used a shared GitHub repository to store the source code, experiment configurations and technical documentation produced during the project. The first version of the repository contained the baseline RAG pipeline. When we reorganised the system around the Retriever, Selector and Generator, we defined the data exchanged between the modules through the shared contracts and common pipeline described in Section 3.3. The implementation and documentation for each module were maintained in separate areas of the repository. Each module group developed and tested its methods within its corresponding area. After a method was updated, we ran the relevant module tests and then connected the updated component to the common pipeline for smoke and integration testing. We resolved interface problems before using the new version in subsequent experiments.

We initially used week- or task-specific branches while developing the baseline methods. After the modular reorganisation, `refactor/three-module-baseline` became the shared branch for combining updates from the three module groups. We used tags to record the fixed versions associated with the main development and evaluation stages:

- **Original RAG pipeline.** Snapshot of the basic RAG pipeline.
Tag: `legacy-before-three-module-reset-2026-07-13`.
- **Three-module development.** Shared integration branch for Retriever, Selector and Generator updates.
Branch: `refactor/three-module-baseline`.
- **Module evaluation.** Fixed versions used in the Selector and Generator evaluations.
Selector tag: `r013-final-2026-08-10`
Generator tag: `generator-frozen-g9-qampari-2026-08-10`.
- **Research archive.** Complete experimental development history.
Tag: `research-archive-2026-08-25`.
- **Final submission.** Final integrated version associated with this report.
Tag: `v1.0.0-dissertation`.

4 Experimental Evaluation and Results

4.1 Evaluation Overview

The evaluation was organised at two levels. Module-level experiments examined the behaviour of individual components, while end-to-end experiments compared the integrated pipeline with complete RAG baselines and measured the contribution of each module through ablation. Table 4.1 summarises the evaluation questions and datasets used in this chapter.

Evaluation	Evaluation question	Datasets
Retriever	How do retrieval signals and fusion strategies affect the ranking and coverage of answer-supporting evidence?	SciFact; KILT-NQ; 2WikiMultiHopQA
Selector	How effectively does the Selector remove harmful evidence without losing required evidence or disrupting multi-hop evidence chains?	NIAH; 2WikiMultiHopQA
Generator	How do generation and verification strategies affect answer coverage, correctness, and citation precision and recall?	ALCE-ASQA; ALCE-QAMPARI
End-to-end comparison	How does the complete system perform relative to established RAG baselines in answer reliability and evidence traceability?	KILT-NQ; KILT-TriviaQA; ALCE-ASQA
Module ablation	How does replacing or removing each module affect end-to-end performance?	KILT-NQ; KILT-TriviaQA; ALCE-ASQA

Table 4.1: Overview of the experimental evaluation.

4.2 Module-Level Evaluation and Results

4.2.1 Retriever Evaluation and Results

Evaluation Objective

The Retriever evaluation assessed the effectiveness of eight retrieval configurations in ranking and recovering answer-supporting evidence. Performance was measured in terms of the rank of relevant evidence and its coverage within the retrieved results across three development benchmarks. The evaluation was used to identify the most reliable configuration for the final Retriever.

Experimental Setup

We evaluated eight Retriever configurations on three development benchmarks: SciFact ($n = 300$ queries), KILT-NQ ($n = 2000$), and 2WikiMultiHopQA ($n = 2000$). These configurations fell into three groups: two sparse retrievers (BM25 and Strong-BM25), three dense or hybrid retrievers (Granite-dense, Hybrid-RRF, and Hybrid-Convex), and three query-modification retrievers (Query2Doc, HyDE, and Decompose).

Each configuration returned the top 50 candidates for every query under the same evaluation conditions. The first ten candidates determined Recall@10 and formed the evidence budget passed to the next pipeline stage. To isolate retrieval effects, the same fixed Top-K Selector and extractive Generator processed the outputs of all configurations.

Evaluation Metrics

We used MRR and Recall@10 to evaluate ranking quality and evidence coverage. MRR calculates the mean reciprocal rank of the highest-ranked relevant document, with a higher value indicating that the Retriever places the first relevant result earlier. Recall@10 measures the proportion of relevant documents included in the first ten results. All queries within each benchmark contained valid relevance annotations and contributed to the reported metrics.

For pairwise comparisons, we matched results by query ID and applied a two-sided sign-flip randomisation test with 10,000 draws. When none of the sampled permutations produced

4 Experimental Evaluation and Results

a difference as extreme as the observed result, we report $p < 0.0001$.

Main Results

	SciFact ($n = 300$)		KILT-NQ ($n = 2000$)		2Wiki ($n = 2000$)	
Variant	MRR $\uparrow$	R@10 $\uparrow$	MRR $\uparrow$	R@10 $\uparrow$	MRR $\uparrow$	R@10 $\uparrow$
bm25	0.6078	0.7562	0.8157	0.7598	0.9434	0.7151
strong-bm25	0.6105	0.7604	0.8153	0.7476	0.9580	0.7222
granite-dense	0.7179	0.8627	0.9327	0.7863	0.9794	0.7418
hybrid-rrf	0.7066	0.8627	0.8866	0.8220	0.9828	0.7460
hybrid-convex	0.7233	0.8572	0.9213	0.8489	0.9871	0.7494
query2doc	0.6485	0.8241	0.8416	0.7760	0.9356	0.7331
hyde	0.7001	0.8617	0.9055	0.7734	0.9512	0.7361
decompose	0.5584	0.7094	0.7682	0.7050	0.5702	0.5491

Table 4.2: MRR and Recall@10 of eight Retriever configurations across three development benchmarks.

	SciFact ($n = 300$)		KILT-NQ ($n = 2000$)		2Wiki ($n = 2000$)	
Configuration vs baseline	Δ MRR $\uparrow$	p	Δ MRR $\uparrow$	p	Δ MRR $\uparrow$	p
hybrid-rrf vs strong-bm25	+0.0961	<0.0001	+0.0713	<0.0001	+0.0248	<0.0001
hybrid-convex vs granite-dense	+0.0054	0.6889	-0.0114	0.0047	+0.0076	<0.0001
strong-bm25 vs bm25	+0.0027	0.7792	-0.0004	0.9181	+0.0146	<0.0001
query2doc vs strong-bm25	+0.0380	0.0265	+0.0262	<0.0001	-0.0224	<0.0001
hyde vs granite-dense	-0.0178	0.1476	-0.0272	<0.0001	-0.0282	<0.0001
decompose vs strong-bm25	-0.0521	0.0018	-0.0472	<0.0001	-0.3878	<0.0001

Table 4.3: Paired MRR differences between Retriever configurations and their corresponding baselines.

Analysis and Discussion

Tables 4.2 and 4.3 indicate that the choice of retrieval signal had a greater effect than tuning the sparse baseline. Strong-BM25 remained close to BM25 on SciFact and KILT-NQ, whereas Granite-dense produced a much larger improvement. On SciFact, MRR increased from 0.6105 to 0.7179 and Recall@10 from 0.7604 to 0.8627; on KILT-NQ, the corresponding values increased from 0.8153 to 0.9327 and from 0.7476 to 0.7863. The

4 Experimental Evaluation and Results

same direction appeared on 2WikiMultiHopQA, although the gains were smaller because the sparse baseline already ranked the first relevant document highly. Semantic retrieval therefore improved both the early placement and the Top-10 coverage of relevant evidence more consistently than further adjustment of BM25.

Combining the sparse and dense signals changed the balance between ranking and coverage. Hybrid-RRF improved MRR over Strong-BM25 on all three benchmarks, with gains of 0.0961, 0.0713, and 0.0248, respectively, and $p < 0.0001$ throughout. Recall@10 also increased on each dataset, including from 0.7476 to 0.8220 on KILT-NQ. Hybrid-Convex achieved the highest value in four of the six reported columns, but its advantage depended on the dataset and metric. On KILT-NQ, for example, it increased Recall@10 to 0.8489 while reducing MRR from the Granite-dense result of 0.9327 to 0.9213. Hybrid-RRF did not lead every individual column, but it offered a more consistent improvement over the sparse baseline and avoided a dataset-specific fusion weight, making it the more suitable default configuration.

Query modification did not strengthen this pattern. Query2Doc improved retrieval on SciFact and KILT-NQ but reduced MRR on 2WikiMultiHopQA, while HyDE remained below Granite-dense across all three benchmarks. Decomposition caused the clearest decline, reducing 2WikiMultiHopQA MRR from the Strong-BM25 result of 0.9580 to 0.5702. Since none of these methods improved ranking and coverage consistently, they were excluded from the final Retriever.

Module-Level Finding

Retrieval signals had a direct effect on both the ranking and coverage of answer-supporting evidence. Sparse tuning produced only limited changes, while the Granite dense signal consistently placed relevant evidence earlier and retained more of it within the first ten results. Fusion provided an additional improvement, although the two strategies behaved differently: Hybrid-Convex achieved stronger results in several individual settings, whereas Hybrid-RRF delivered more consistent gains across benchmarks without requiring a dataset-specific weight. This balance of ranking performance, evidence coverage, and cross-dataset consistency led to the selection of Hybrid-RRF as the final Retriever configuration.

4.2.2 Selector Evaluation and Results

Evaluation Objective

The final NLI-aware Protect-Harm Selector was evaluated against three evidence-level objectives:

1. **Harmful-evidence removal:** whether the Selector removed passages labelled as harmful.
2. **Required-evidence retention:** whether it retained the passages required to answer the query.
3. **Multi-hop chain preservation:** whether it preserved the complete set of supporting passages needed for multi-hop questions.

Experimental Setup

NIAH was used to measure harmful-evidence removal and required-evidence retention, while 2WikiMultiHopQA assessed the preservation of multi-hop supporting evidence. For each query, the Retriever returned a ranked pool of 20 candidate passages. We used the first ten as the common input to the TopK10 baseline and all Selector variants.

For the final evaluation, we configured the NLI-based Selector with $q = 0.99$ as the threshold-calibration quantile and $\text{cap} = 2$ as the maximum number of deletions per query. We trained and evaluated this configuration independently using Seeds 13 and 42.

To distinguish targeted evidence selection from the effect of simply reducing the number of passages, we included two non-learned controls for Seed 13. Both controls followed the Selector’s deletion count for each query. Random deletion selected passages from the TopK10 input across 100 repetitions, whereas bottom-rank deletion removed the lowest-ranked passages.

Evaluation Metrics

We used the following metrics to evaluate harmful-evidence removal and the preservation of required evidence.

4 Experimental Evaluation and Results

- $\Delta H \uparrow$: Harmful-evidence reduction measures the proportion of labelled harmful evidence removed from the original TopK10 context.
- $P_D \uparrow$: Deletion precision measures the proportion of removed passages that were labelled harmful. A higher value indicates more selective removal of harmful passages.
- $L_R^N \downarrow$ and $L_R^W \downarrow$: Evidence-recall loss measures the loss of required evidence on NIAH and supporting evidence on 2WikiMultiHopQA relative to TopK10. A value of 0 % indicates that all labelled required or supporting evidence was retained.
- $L_C^N \downarrow$ and $L_C^W \downarrow$: Evidence-chain loss measures the loss of complete evidence chains on NIAH and 2WikiMultiHopQA relative to TopK10. A value of 0 % indicates that every complete labelled evidence chain was preserved.
- $\bar{d}$: Mean deletions per query reports the average number of passages removed for each query. This descriptive measure has no preferred direction.
- **Notation:** N denotes NIAH and W denotes 2WikiMultiHopQA. $\uparrow$ means higher is better, whereas $\downarrow$ means lower is better. % denotes percentage differences, and n/a means that the metric was not evaluated for the control.

Evidence-Level Results

Configuration	$\Delta H \uparrow$	$P_D \uparrow$	$L_R^N \downarrow$	$L_C^N \downarrow$	$L_R^W \downarrow$	$L_C^W \downarrow$	$\bar{d}$
Protect–Harm, Seed 13	12.95%	75.44%	0%	0%	0%	0%	0.0656
Protect–Harm, Seed 42	13.10%	80.56%	0%	0%	0%	0%	0.0621
Random deletion	1.62%	9.45%	n/a	n/a	n/a	n/a	0.0656
Bottom-rank deletion	0.30%	1.75%	n/a	n/a	n/a	n/a	0.0656

Table 4.4: Evidence-level results for the final Protect–Harm Selector under misleading-evidence stress tests.

Across the two seeds, the Selector removed 12.95% and 13.10% of the labelled harmful evidence, with deletion precision of 75.44% and 80.56%, respectively. The Seed 13 harmful-evidence reduction had a 95% bootstrap confidence interval of 10.34–15.84%. By comparison, the count-matched random and bottom-rank controls removed 1.62% and 0.30% of the harmful evidence, with deletion precision of 9.45% and 1.75%.

4 Experimental Evaluation and Results

Both trained models recorded zero observed required-evidence recall loss and zero evidence-chain loss on NIAH. They also recorded zero supporting-evidence recall loss and zero chain loss on 2WikiMultiHopQA. The average number of passages removed per query was 0.0656 for Seed 13 and 0.0621 for Seed 42. The close agreement between the two seeds indicates consistent, selective evidence-filtering behaviour across the two training runs.

Module-Level Finding

The module-level evaluation showed the intended filtering behaviour across the three evidence-level objectives. The final Selector removed labelled harmful passages more effectively than the count-matched controls, while retaining all labelled required evidence on NIAH and preserving all supporting-evidence chains on 2WikiMultiHopQA. The consistent results across the two seeds further indicate stable evidence-filtering behaviour across independent training runs.

4.2.3 Generator Evaluation and Results

Evaluation Objective

The Generator evaluation assessed the final Verify-and-Label design against three objectives: linking supported claims to valid source evidence, clearly identifying claims without verified support, and preserving answer coverage and correctness.

Experimental Setup

The evaluation used two ALCE answer-generation benchmarks, ASQA [15] and QAMPARI [16], sampling 400 queries from each. ASQA served as the calibration set, while QAMPARI provided a held-out evaluation on a separate dataset. For each query, every Generator configuration received the same five passages returned by the GTR Retriever.

Five configurations were evaluated on ASQA. Generation-time Citation served as the baseline, while Delete Filter represented the deletion-based design. Label, Original Contract and Label, Gate Engaged captured two intermediate stages of the labelling design, and Verify-and-Label represented the final Generator. The QAMPARI evaluation compared the final configuration with Generation-time Citation, Delete Filter and Label, Gate Engaged.

4 Experimental Evaluation and Results

Within each dataset, all configurations used the same queries, evidence, model and decoding settings, with the random seed fixed at 13.

Evaluation Metrics

We used five metrics to evaluate answer preservation and citation quality. MiniCheck [13], a separate evaluation model, judged whether the cited passages supported the corresponding answer sentences.

- **Answer Coverage $\uparrow$:** The proportion of queries for which the Generator produced a non-empty answer.
- **Correctness $\uparrow$:** ASQA uses STR-EM to measure how much of the reference answer content appears in the generated answer. QAMPARI uses containment-based answer recall. Because the definitions differ, correctness is interpreted only within each dataset.
- **Citation Precision, Cited-Sample $\uparrow$:** The proportion of citations that support their associated answer sentences, averaged only over answers containing at least one citation.
- **Citation Precision, ALCE-Standard $\uparrow$:** The same support measure, with a score of zero assigned to an answered query that contains no citation.
- **Citation Recall $\uparrow$:** The proportion of answer sentences supported by their cited passages.
- **Notation:** $\uparrow$ means higher is better. Δ denotes the mean paired difference between two configurations, with positive values favouring the first. The tables also report the randomisation-test p -value and the number of valid query pairs, n . Bootstrap resampling provides the 95% confidence intervals.

ASQA Calibration Results

4 Experimental Evaluation and Results

Paired comparison	Stat.	Answer coverage ↑	Correctness ↑	Citation precision (cited-sample) ↑	Citation precision (ALCE-standard) ↑	Citation recall ↑
Verify-and-Label vs. Generation-time	Δ (%)	-1.52	+0.15	+19.13	+11.92	+7.11
Citation	p	0.386	0.892	< 0.001	< 0.001	0.009
	n	395	395	294	346	346
Delete Filter vs. Generation-time	Δ (%)	-28.61	-5.94	+17.12	+17.12	+13.06
Citation	p	< 0.001	< 0.001	< 0.001	< 0.001	< 0.001
	n	395	395	245	245	245
Verify-and-Label vs. Delete Filter	Δ (%)	+27.09	+6.09	+2.24	+2.24	+0.11
	p	< 0.001	< 0.001	0.238	0.238	0.970
	n	395	395	253	253	253
Verify-and-Label vs. Label, Gate Engaged	Δ (%)	+12.66	+4.41	-0.46	+0.59	+0.43
	p	< 0.001	< 0.001	0.698	0.624	0.720
	n	395	395	251	310	310
Label, Gate Engaged vs. Label, Original	Δ (%)	+14.94	+2.13	0.00	0.00	0.00
Contract	p	< 0.001	< 0.001	1.000	1.000	1.000
	n	395	395	251	251	251

Table 4.5: Paired comparisons between Generator configurations on ASQA.

Compared with Generation-time Citation, Verify-and-Label produced substantially stronger citation results on ASQA. Cited-sample citation precision increased by 19.13 %, ALCE-standard precision by 11.92 %, and citation recall by 7.11 %. The smaller changes in answer coverage (-1.52 %, $p = 0.386$) and correctness (+0.15 %, $p = 0.892$) were not statistically significant.

The difference between labelling and deletion was most visible in answer preservation. Verify-and-Label recovered 27.09 % of answer coverage and 6.09 % of correctness relative to Delete Filter, with both differences reaching $p < 0.001$. Citation precision and recall changed by only +0.11 to +2.24 %, and none of these differences was statistically significant. On ASQA, labelling therefore retained substantially more answer content than deletion while maintaining comparable citation quality.

4 Experimental Evaluation and Results

QAMPARI Held-out Results

Paired comparison	Stat.	Answer coverage ↑	Correctness ↑	Citation precision (cited-sample) ↑	Citation precision (ALCE-standard) ↑	Citation recall ↑
Verify-and-Label vs. Generation-time Citation	Δ (%)	-1.57	-1.05	+26.85	+14.67	+9.29
	p	0.469	0.025	< 0.001	< 0.001	0.010
	n	382	382	233	280	280
Verify-and-Label vs. Delete Filter	Δ (%)	+18.85	+1.02	+4.35	+3.90	+2.58
	p	< 0.001	< 0.001	0.039	0.073	0.249
	n	382	382	227	228	228
Verify-and-Label vs. Label, Gate Engaged	Δ (%)	+6.54	+0.68	-0.66	-0.16	-0.10
	p	< 0.001	< 0.001	0.350	0.824	0.874
	n	382	382	220	275	275

Table 4.6: Paired comparisons between Generator configurations on QAMPARI.

On QAMPARI, Verify-and-Label again improved citation traceability relative to Generation-time Citation: cited-sample precision increased by 26.85 %, ALCE-standard precision by 14.67 %, and citation recall by 9.29 %. It also preserved more answer content than Delete Filter, increasing coverage by 18.85 % and correctness by 1.02 %. However, correctness was 1.05 % lower than Generation-time Citation, and the corresponding coverage comparison was inconclusive because claim-splitting failures were more frequent on QAMPARI.

Module-Level Finding

The Generator evaluation showed that Verify-and-Label consistently improved the traceability and reliability of generated answers across ASQA and QAMPARI. Compared with generation-time citations, the final configuration achieved higher citation precision and recall on both datasets. Labelling claims without verified support also preserved substantially more answer content than deleting them, while maintaining comparable citation quality. However, the held-out results did not establish a general improvement in answer quality: QAMPARI showed a small decline in correctness, and its coverage comparison was affected by claim-splitting failures. The final Generator was therefore selected for its stronger citation traceability and reliability, and lower content loss, rather than for an improvement in answer correctness.

4.3 End-to-End Evaluation and Results

4.3.1 Experimental Setup

Evaluation Objectives

The end-to-end evaluation addressed two questions: how the complete system compared with the baseline RAG systems in answer reliability and evidence traceability, and how each module contributed to the performance of the integrated pipeline.

Common Experimental Setup

The final evaluation compared the proposed system with four RAG baselines—BM25 RAG, Hybrid RAG, Granite Rerank RAG and Provence RAG—on KILT-NQ, KILT-TriviaQA and ALCE-ASQA. The tables report the proposed-system results as the mean $\pm$ sample standard deviation across runs using Seeds 13, 42 and 73, alongside a point estimate from one complete run for each deterministic baseline. Module configurations and evaluation settings remained fixed throughout the final comparison, and all results in the system comparison and ablation tables are expressed as percentages.

Evaluation Metrics

The six metrics capture factual coverage, evidence support, citation quality and response behaviour. Table 4.7 defines each measure before the system-level results are presented.

Metric	Meaning	Direction
RFC	Reference facts correctly expressed in the answer	↑
VRFC	Reference facts both correctly expressed and supported by a valid citation	↑
UCR	Factual claims unsupported by the presented evidence	↓
CP	Citation links that support their associated claims	↑
CR	Factual claims with at least one valid supporting citation	↑
RR	Queries receiving a substantive answer rather than an abstention	↑

Table 4.7: End-to-end evaluation metrics (↑: higher is better; ↓: lower is better).

4 Experimental Evaluation and Results

4.3.2 Main System Comparison

Tables 4.8–4.10 report the complete-system results for the three evaluation datasets (all values in %).

System	RFC $\uparrow$	VRFC $\uparrow$	UCR $\downarrow$	CP $\uparrow$	CR $\uparrow$	RR $\uparrow$
BM25 RAG	47.50	26.25	30.91	40.56	44.45	94.25
Hybrid RAG	60.75	37.50	17.34	49.60	55.02	96.50
Granite Rerank RAG	64.00	41.25	18.02	49.30	57.26	96.75
Providence RAG	60.50	40.00	19.91	50.30	57.27	95.50
Ours	51.08 ± 0.58	38.00 ± 0.43	18.80 ± 0.98	60.00 ± 1.25	61.10 ± 1.19	92.17 ± 1.66

Table 4.8: Main system comparison: (a) KILT–NQ.

System	RFC $\uparrow$	VRFC $\uparrow$	UCR $\downarrow$	CP $\uparrow$	CR $\uparrow$	RR $\uparrow$
BM25 RAG	81.00	43.50	21.28	42.13	47.13	92.75
Hybrid RAG	84.00	49.25	20.52	47.07	52.83	95.00
Granite Rerank RAG	88.50	54.75	16.37	47.91	57.75	96.00
Providence RAG	82.25	45.75	26.82	41.84	49.46	93.50
Ours	78.58 ± 1.94	54.67 ± 0.80	18.41 ± 1.52	54.10 ± 0.90	57.56 ± 1.02	88.42 ± 2.43

Table 4.9: Main system comparison: (b) KILT–TriviaQA.

System	RFC $\uparrow$	VRFC $\uparrow$	UCR $\downarrow$	CP $\uparrow$	CR $\uparrow$	RR $\uparrow$
BM25 RAG	24.61	14.91	27.12	38.33	43.54	90.00
Hybrid RAG	30.00	16.60	20.69	42.90	49.34	94.25
Granite Rerank RAG	33.33	21.42	15.81	47.75	57.42	97.00
Providence RAG	28.67	18.22	25.71	46.31	54.56	91.75
Ours	23.20 ± 0.67	18.28 ± 0.25	21.04 ± 0.83	57.21 ± 0.87	58.46 ± 1.02	90.25 ± 1.64

Table 4.10: Main system comparison: (c) ALCE–ASQA.

Cross-Dataset Results

Across all three datasets, our system achieved the highest Citation Precision (CP), reaching 60.00% on KILT–NQ, 54.10% on KILT–TriviaQA and 57.21% on ALCE–ASQA. These scores exceeded the strongest baseline by 9.70%, 6.19% and 9.46%, respectively. The consistent improvement across different question-answering settings shows that the sources

4 Experimental Evaluation and Results

cited in our answers more often supported the claims they accompanied.

Citation Recall (CR) remained similarly strong. Our system reached 61.10% on KILT-NQ and 58.46% on ALCE-ASQA, improving on the strongest baselines by 3.83% and 1.04%. On KILT-TriviaQA, its score of 57.56% was only 0.19% below the highest result. Together, the precision and recall results show that citations were more accurate where they appeared, while a comparable or larger share of the answer remained connected to valid source evidence. This gave the generated answers more reliable evidence and clearer traceability back to the supporting sources.

The remaining gap lies in the completeness of the final answers. Compared with Granite Rerank RAG, our system was 9.92–12.92% lower in Reference Fact Coverage (RFC) and 4.58–7.58% lower in Response Rate (RR) across the three datasets. Its Unsupported Claim Rate (UCR) also remained 1.46%, 2.04% and 5.23% above the lowest baseline on KILT-NQ, KILT-TriviaQA and ALCE-ASQA, respectively. The complete system therefore still needs to cover more of the required answer content and reduce unsupported claims while preserving the reliability and traceability reflected in its citation results.

4.3.3 Module Ablation

Ablation Setup

The ablation used the frozen Seed 13 Full system as the common reference and changed one module in each comparison. *w/ BM25 Retriever* used BM25 in place of the full Retriever; *w/o Selector / Keep-all Top-10* passed all ten retrieved passages directly to the Generator; and *w/ Direct Generator* generated the answer without the grounded generation procedure. The remaining modules and evaluation conditions were kept unchanged.

Tables 4.11–4.13 report the module ablation results for the three evaluation datasets. All values are reported as percentages.

4 Experimental Evaluation and Results

Configuration	RFC $\uparrow$	VRFC $\uparrow$	UCR $\downarrow$	CP $\uparrow$	CR $\uparrow$	RR $\uparrow$
Full	50.75	37.75	19.31	60.96	62.00	93.25
w/ BM25 Retriever	38.25	24.25	32.16	46.00	46.63	91.50
w/o Selector / Keep-all Top-10	50.25	37.00	19.25	59.96	61.00	92.25
w/ Direct Generator	60.50	37.75	17.74	49.25	55.20	97.00

Table 4.11: Module ablation results: (a) KILT–NQ.

Configuration	RFC $\uparrow$	VRFC $\uparrow$	UCR $\downarrow$	CP $\uparrow$	CR $\uparrow$	RR $\uparrow$
Full	80.75	55.25	20.11	54.83	57.88	90.50
w/ BM25 Retriever	74.75	52.25	22.47	52.29	55.58	88.25
w/o Selector / Keep-all Top-10	80.75	55.25	20.11	54.83	57.88	90.50
w/ Direct Generator	84.00	49.00	20.52	47.03	52.33	95.00

Table 4.12: Module ablation results: (b) KILT–TriviaQA.

Configuration	RFC $\uparrow$	VRFC $\uparrow$	UCR $\downarrow$	CP $\uparrow$	CR $\uparrow$	RR $\uparrow$
Full	22.60	17.99	21.95	56.83	58.00	91.75
w/ BM25 Retriever	20.25	14.98	30.45	47.75	48.38	89.25
w/o Selector / Keep-all Top-10	22.75	17.93	20.94	56.21	57.50	90.25
w/ Direct Generator	29.75	15.90	20.88	41.67	48.21	94.75

Table 4.13: Module ablation results: (c) ALCE–ASQA.

Contribution of Individual Modules

Retriever. Compared with BM25, the full Retriever improved all six metrics across the three datasets. The largest differences appeared on KILT–NQ, where RFC increased from 38.25% to 50.75% and CR from 46.63% to 62.00%. On ALCE–ASQA, UCR decreased from 30.45% to 21.95%, accompanied by gains of approximately 9% in both CP and CR. These results show that the Retriever strengthened the evidence base passed to the downstream modules.

4 Experimental Evaluation and Results

Selector. Changes on the ordinary end-to-end datasets remained small. The Full and Keep-all Top-10 configurations produced identical results on KILT–TriviaQA, while CP and CR each increased by 1.00% on KILT–NQ. In the misleading-evidence stress tests, however, the Selector removed 12.95–13.10% of the labelled harmful evidence with deletion precision of 75.44–80.56%, while preserving all evaluated required evidence and multi-hop chains. The contrast between these settings characterises the Selector as a risk-triggered evidence-filtering module.

Generator. The grounded Generator improved CP by 7.80–15.16% and CR by 5.55–9.79% across the three datasets. Direct generation produced higher RFC and RR, with differences of 3.25–9.75% and 3.00–4.50%, respectively. The grounded Generator’s clearest contribution was therefore the valid support and traceability of generated claims.

End-to-End Finding

Taken together, the end-to-end and ablation results show how the three modules support reliable and traceable evidence use at different stages of the pipeline. The Retriever establishes the evidence base, the Selector provides targeted filtering when risky evidence is present, and the grounded Generator maintains the valid connection between the final answer and its supporting sources.

5 Conclusion

5.1 Overall Conclusion

This project addressed the needle-in-a-haystack problem by developing and evaluating an integrated Granite-based RAG system that maintains a traceable path from source documents to the final response. Guided by Reliable Evidence Flow, we treated evidence-grounded answering as an end-to-end process: relevant information must not only be found, but also remain available, be used appropriately, and retain a clear connection to its source. We therefore developed methods for evidence discovery, context control, and source-grounded answer generation and integrated them within a common research prototype.

We then evaluated these methods through module-level experiments, comparisons with RAG baselines, and ablation experiments on the integrated system. These evaluations examined the reliability and traceability of the complete pipeline and clarified how its stages contribute to the final response when combined. Together, the work shows that reliable evidence-grounded answering is a system-level capability produced through the coordinated discovery, control, and grounding of evidence. Reliable Evidence Flow therefore reframes the needle-in-a-haystack task as the management of a complete evidence path, linking the discovery of relevant information to its controlled use and traceable attribution in the final answer.

5.2 Critical Evaluation

Reliable Evidence Flow strengthened both reliability and traceability across the complete RAG process. Through module-level experiments, comparisons with RAG baselines, and

5 Conclusion

ablation experiments, we evaluated the proposed methods and their contribution to the integrated system. The results provided clear evidence for both dimensions of the project aim: reliability was strengthened by preserving required evidence and controlling misleading information, while traceability was improved through clearer and more verifiable links between final claims and their supporting sources. Together, these findings demonstrate the value of managing evidence as a connected flow from source documents to the final answer.

Viewed retrospectively, the clearest design lesson was that uncertainty should not be treated as sufficient reason for irreversible deletion. Early Selector designs sometimes removed required evidence when apparent conflicts reflected uncertainty, while the Generator’s Delete Filter improved citation quality at the cost of answer coverage and correctness. The final system therefore adopts asymmetric evidence control: deletion is reserved for high-risk evidence, while claims without verified support remain visible through explicit labels.

The main remaining limitation concerns the completeness of the final response. Some measures of answer coverage and unsupported content were less consistent, indicating that the complete system must continue to balance the preservation of useful information with evidence validation. Evaluation also remained at benchmark and prototype scale, so performance across organisation-scale collections and live enterprise workloads has not yet been established. Overall, the prototype establishes a credible system-level approach to reliable and traceable evidence use, while further evaluation is needed to determine whether these properties can be maintained in changing enterprise environments.

5.3 Future Work

Our results point to a broader next step for Reliable Evidence Flow: moving from a one-way pipeline towards an adaptive, feedback-aware framework that manages evidence as a system-level process. The experiments show that improving one stage does not automatically improve the final response; reliability emerges from how the system retrieves, preserves, uses, and verifies evidence across the complete answer-forming process. Future work can therefore focus on enabling the system to assess the condition of the evidence flow as a whole and adjust its intervention accordingly, rather than relying only on fixed decisions

5 Conclusion

within individual stages. Feedback from the final answer, including both answer quality and the quality of evidence use, could provide learning signals for earlier evidence decisions and connect evidence control to its end-to-end consequences. Investigating this relationship will require broader training data, greater computational resources, and more research time. Evaluation should also move beyond static benchmarks into realistic enterprise environments, where document versions evolve, sources conflict, and access permissions change the available evidence. The central question is whether Reliable Evidence Flow can remain reliable and traceable not only within a single pipeline run, but as its evidence base changes over time.

5.4 Contribution Statement

The team jointly developed the initial RAG pipeline, establishing a shared technical foundation for the project. After discussing the project direction with the IBM client, all members reviewed and discussed relevant literature. Drawing on this shared review, the team established Reliable Evidence Flow as the central research direction and organised the subsequent research around three parallel modules.

The team divided the module and demonstration responsibilities as follows:

- **Retriever:** Mengxiao Wu and Zenan Wu.
- **Selector:** Jiaxi You and Weikai Mao.
- **Generator:** Ming Wei and Zhanyu Xu.
- **System integration:** Jiaxi You.
- **Full-Stack Demonstration:** Frida Chen.

Within each module, the assigned members led the full research process, from module-specific literature review and method design to implementation, iterative experimentation, and analysis of the results. Jiaxi You integrated the three modules into the complete system and led its overall experimental evaluation. The full team jointly prepared and reviewed the final report.

Appendix A

Technical Dependencies

We implemented the experimental pipeline in Python 3.11. Table A.1 summarises the packages used and their corresponding Python import names.

PyPI package	Python import	Usage
<code>pydantic</code>	<code>pydantic</code>	Configuration and data validation.
<code>torch</code>	<code>torch</code>	Model inference and training.
<code>transformers</code>	<code>transformers</code>	Granite and NLI model loading.
<code>sentence-transformers</code>	<code>sentence_transformers</code>	Dense retrieval and Cross-Encoder models.
<code>peft</code>	<code>peft</code>	Parameter-efficient fine-tuning and LoRA.
<code>ir-datasets</code>	<code>ir_datasets</code>	Information-retrieval benchmark loading.
<code>docling</code>	<code>docling</code>	PDF and Office document processing.
<code>docling-core[chunking]</code>	<code>docling_core</code>	Document structures and chunking support.
<code>pillow</code>	<code>PIL</code>	Image loading and preprocessing.
<code>datasets</code>	<code>datasets</code>	Hugging Face dataset loading.
<code>sentencepiece</code>	<code>sentencepiece</code>	Tokenisation for compatible NLI models.
<code>spacy</code>	<code>spacy</code>	Optional entity extraction.
<code>pytest</code>	<code>pytest</code>	Automated testing.
<code>ruff</code>	Command-line tool	Linting and code-quality checks.
<code>mypy</code>	Command-line tool	Static type checking.
<code>build</code>	Command-line tool	Python package construction.
<code>hatchling</code>	Build backend	Wheel and source-package generation.

Table A.1: Python dependencies

Only `pydantic` is required by the lightweight core installation. Packages for model execution, retrieval, document ingestion, evidence selection, and development are organised into optional dependency groups. This keeps the core Retriever–Selector–Generator pipeline lightweight while allowing additional components to be installed when required.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, Curran Associates, Inc., 2020, pp. 9459–9474. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [2] J. Zhang, H. Zhang, D. Zhang, Y. Liu, and S. Huang, “End-to-end beam retrieval for multi-hop question answering,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Mexico City, Mexico: Association for Computational Linguistics, 2024, pp. 1718–1731. DOI: 10.18653/v1/2024.naacl-long.96. [Online]. Available: <https://aclanthology.org/2024.naacl-long.96/>.
- [3] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024. DOI: 10.1162/tac1_a_00638. [Online]. Available: <https://aclanthology.org/2024.tacl-1.9/>.
- [4] O. Yoran, T. Wolfson, O. Ram, and J. Berant, “Making retrieval-augmented language models robust to irrelevant context,” in *Proceedings of the Twelfth International Conference on Learning Representations*, OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=ZS4m74kZpH>.
- [5] T. Gao, H. Yen, J. Yu, and D. Chen, “Enabling large language models to generate text with citations,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Singapore: Association for Computational Linguistics,

References

- 2023, pp. 6465–6488. DOI: 10.18653/v1/2023.emnlp-main.398. [Online]. Available: <https://aclanthology.org/2023.emnlp-main.398/>.
- [6] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, Online: Association for Computational Linguistics, 2020, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550. [Online]. Available: <https://aclanthology.org/2020.emnlp-main.550/>.
- [7] S. Bruch, S. Gai, and A. Ingber, “An analysis of fusion functions for hybrid retrieval,” *ACM Transactions on Information Systems*, vol. 42, no. 1, 20:1–20:35, 2024. DOI: 10.1145/3596512. [Online]. Available: <https://doi.org/10.1145/3596512>.
- [8] Y. Yu, W. Ping, Z. Liu, B. Wang, J. You, C. Zhang, M. Shoenybi, and B. Catanzaro, “RankRAG: Unifying context ranking with retrieval-augmented generation in LLMs,” in *Advances in Neural Information Processing Systems*, vol. 37, Curran Associates, Inc., 2024, pp. 121 156–121 184. DOI: 10.52202/079017–3850. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/db93ccb6cf392f352570dd5af0a223d3-Abstract-Conference.html.
- [9] D. Lee, Y. Jo, H. Park, and M. Lee, “Shifting from ranking to set selection for retrieval augmented generation,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Vienna, Austria: Association for Computational Linguistics, 2025, pp. 17 606–17 619. DOI: 10.18653/v1/2025.acl-long.861. [Online]. Available: <https://aclanthology.org/2025.acl-long.861/>.
- [10] Z. Shen, B. Imana, T. Wu, C. Xiang, P. Mittal, and A. Korolova, “ReliabilityRAG: Effective and provably robust defense for RAG-based web-search,” in *Advances in Neural Information Processing Systems*, vol. 38, Curran Associates, Inc., 2025, pp. 45 662–45 702. DOI: 10.52202/085713–1522. arXiv: 2509.23519. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/hash/41457d56a2fdd20b4c072190290a549b-Abstract-Conference.html.
- [11] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, and T. Schuster, “Conformal risk control,” in *Proceedings of the Twelfth International Conference on Learning*

References

- Representations*, OpenReview, 2024. [Online]. Available: <https://openreview.net/forum?id=33XGfHLtZg>.
- [12] A. Asai, M. Gardner, and H. Hajishirzi, “Evidentiality-guided generation for knowledge-intensive NLP tasks,” in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Seattle, United States: Association for Computational Linguistics, 2022, pp. 2226–2243. DOI: 10.18653/v1/2022.naacl-main.162. [Online]. Available: <https://aclanthology.org/2022.naacl-main.162/>.
- [13] L. Tang, P. Laban, and G. Durrett, “MiniCheck: Efficient fact-checking of LLMs on grounding documents,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Miami, Florida, USA: Association for Computational Linguistics, 2024, pp. 8818–8847. DOI: 10.18653/v1/2024.emnlp-main.499. [Online]. Available: <https://aclanthology.org/2024.emnlp-main.499/>.
- [14] O. Honovich, R. Aharoni, J. Herzig, H. Taitelbaum, D. Kukliansy, V. Cohen, T. Scialom, I. Szpektor, A. Hassidim, and Y. Matias, “TRUE: Re-evaluating factual consistency evaluation,” in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Seattle, United States: Association for Computational Linguistics, 2022, pp. 3905–3920. DOI: 10.18653/v1/2022.naacl-main.287. [Online]. Available: <https://aclanthology.org/2022.naacl-main.287/>.
- [15] I. Stelmakh, Y. Luan, B. Dhingra, and M.-W. Chang, “ASQA: Factoid questions meet long-form answers,” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, 2022, pp. 8273–8288. DOI: 10.18653/v1/2022.emnlp-main.566. [Online]. Available: <https://aclanthology.org/2022.emnlp-main.566/>.
- [16] S. Amouyal, T. Wolfson, O. Rubin, O. Yoran, J. Herzig, and J. Berant, “QAMPARI: A benchmark for open-domain questions with many answers,” in *Proceedings of the Third Workshop on Natural Language Generation, Evaluation, and Metrics (GEM)*, Singapore: Association for Computational Linguistics, 2023, pp. 97–110. [Online]. Available: <https://aclanthology.org/2023.gem-1.9/>.